MEMORIA DEL PROYECTO

GESTOR DE INCIDENCIAS

DESARROLLO DE APLICACIONES WEB

AUTORES:

ANGEL BELTRAN MOLINA

SAMUEL ECIJA RODRIGUEZ

PROFESOR COORDINADOR:

FRANCISCO NAVARRO VELASCO

Índice

1. RESUMEN 2
2. PALABRAS CLAVE 2
3. INTRODUCCION 2
4. ESTADO DEL ARTE/TRABAJOS RELACIONADOS 3
5. ANALISIS DE OBJETIVOS Y METODOLOGIAS 3
Objetivo general 3
Objetivos específicos 3
Metodología 5
6. DISEÑO Y RESOLUCION DEL TRABAJO REALIZADO 5
6.1 Arquitectura MVC 5
6.2 Base de datos 6
6.3 Sistema de autenticación y seguridad 7
6.4 Roles de usuario 9
6.5 Gestión de tickets 10
6.6 Sistema de permisos granular 13
6.7 Panel de Configuración 14
6.8 Notificaciones 15
6.9 Mensajería Interna 16
6.10 Presencia y actividad 16
6.11 Auditoría 16
6.12 Vista de soporte 17
6.13 Dashboard 18
6.14 Interfaz y experiencia de usuario 19
6.15 Tecnologías utilizadas 20
6.16 Dificultades encontradas 20
7. MANUAL DE INSTALACIÓN Y DESPLIEGUE 21
7.1 Requisitos Mínimos del Sistema 21
7.2 Proceso de Instalación Paso a Paso 21
7.3 Guia de Usuario Final 23
8. CONTROL DE CALIDAD Y PRUEBAS 32
8.1. Estructura del proyecto 32
8.2. Pruebas Automatizadas 32
8.3. Pruebas Manuales 32
8.4. Dependencias 33
8.5. Manejo de Errores 33
8.6. Buenas Prácticas 33
9. PRESUPUESTO 34
9.1. Fases 34
10. CONCLUSIONES Y VIAS FUTURAS 35
Lo que nos hubiera gustado hacer y no ha dado tiempo 35
Vías futuras 35
Reflexión personal 35
11. BIBLIOGRAFIA/WEBGRAFIA 36

1. RESUMEN

Este proyecto consiste en el desarrollo de un sistema de gestión de incidencias y soporte técnico orientado a que la empresas o equipos que necesiten llevar un control estructurado de los problemas reportados por sus trabajadores. La aplicación permite registrar clientes y registrar incidencias (tickets), asignarlos a agentes de soporte y hacerles seguimiento por estados y departamentos, y mantener un historial completo de toda la actividad.

El sistema está desarrollado en PHP 8.2 siguiendo el patrón de arquitectura MVC implementado desde cero, sin depender de frameworks externos. Esto nos ha permitido comprender en profundidad el funcionamiento de un framework propio: enrutamiento, controladores base, gestión de vistas y capa de acceso a datos mediante PDO. La base de datos relacional está gestionada con MariaDB 10.4 y el esquema está definido mediante un script de migración versionado.

Entre las funcionalidades más destacadas se encuentran: un sistema de autenticación completo con recuperación de contraseña por email (PHPMailer), un modelo de permisos granular basado en roles con posibilidad de override individual por usuario, un panel kanban para visualizar el estado de los tickets, notificaciones en tiempo real mediante campana, registro de auditoría de todas las acciones del sistema, y un fondo visual animado con WebGL (Three.js) en las vistas de autenticación.

El proyecto se ha planteado con una clara separación entre backend y frontend, de modo que el núcleo MVC actúa como backend que puede exponerse también como API REST para integrarse con un frontend independiente.

2. PALABRAS CLAVE

php, mvc, gestión incidencias, helpdesk, ticketing, servicio técnico, mysql, mariadb, api rest, three.js, phpmailer, permisos, roles, kanban, desarrollo web

3. INTRODUCCION

La idea del proyecto surge de la necesidad real de uno de los autores, que trabaja en una tienda de informática donde, además de la venta, se realiza servicio de reparación de ordenadores, móviles, impresoras y equipos similares. El software que se usaba en la tienda era una aplicación desarrollada en MSDOS que, aunque funcional, estaba completamente desfasada: no permitía notificar al cliente del estado de su reparación, no ofrecía ningún tipo de historial de actividad, y su interfaz resultaba inaccesible para cualquier usuario sin formación específica.

A partir de esa necesidad concreta, decidimos plantear un sistema más amplio: no solo para un taller de reparaciones, sino para cualquier organización que necesite gestionar incidencias internas entre usuarios y un equipo de soporte. Esto hace que la aplicación sea aplicable en entornos de soporte IT corporativo, centros educativos, servicios técnicos o cualquier estructura con departamentos y agentes.

Tras definir la idea, elaboramos un esquema de requisitos, realizamos un diagrama de base de datos inicial, y comenzamos el desarrollo del backend en PHP. La toma de decisiones técnicas durante el proyecto ha sido constante: desde elegir construir nuestro propio framework MVC en lugar de usar uno existente (decisión que justificamos en el apartado de objetivos), hasta

definir un modelo de permisos que fuera lo suficientemente flexible para cubrir todos los casos de uso que identificamos.

4. ESTADO DEL ARTE/TRABAJOS RELACIONADOS

Actualmente existen numerosas herramientas de gestión de incidencias y helpdesk en el mercado. Las más conocidas son:

- Jira Service Management (Atlassian): potente y muy configurable, pero orientado a equipos de desarrollo ágil y con una curva de aprendizaje elevada. Su versión gratuita es limitada.
- Zendesk: plataforma SaaS muy completa para soporte al cliente. De pago, cerrada y sin posibilidad de autoalojamiento.
- osTicket: solución open source de ticketing. Funcional, pero con una interfaz anticuada y poca flexibilidad en la estructura de roles.
- Freshdesk: orientado a atención al cliente. Interfaz moderna, pero de pago para funcionalidades avanzadas.
- GLPI: software de gestión de activos e incidencias muy usado en entornos educativos y corporativos. Muy completo pero pesado y difícil de personalizar.

Ninguna de estas soluciones cumplía con lo que buscábamos: un sistema ligero, autoalojado y adaptable que pudiéramos entender y ampliar en su totalidad al haberlo construido nosotros mismos.

Valor añadido de nuestra propuesta:

- Arquitectura MVC implementada desde cero en PHP
- Sistema de permisos granular por rol con overrides individuales
- Vista Kanban integrada para gestión visual de tickets
- Registro de auditoría completo de todas las acciones del sistema
- Vinculación automática de tickets a clientes por email
- Mensajería interna entre usuarios
- Panel de administración completo desde el propio sistema
- Registro de auditoría de todas las acciones

5. ANALISIS DE OBJETIVOS Y METODOLOGIAS

Objetivo general

Desarrollar una aplicación web de gestión de incidencias con arquitectura MVC propia en PHP, base de datos relacional en MariaDB, sistema de autenticación seguro, gestión de roles y permisos, y notificaciones automáticas por email.

Objetivos específicos

OBJETIVO 1. Implementar una arquitectura MVC desde cero en PHP

Decidimos no usar ningún framework externo (Laravel, Symfony, CodeIgniter) para entender a fondo cómo funciona el patrón MVC. Construimos nuestro propio FrontController que enruta las peticiones por ?controller=&action=, una clase base AppController de la que heredan todos los controladores, y una clase View para el renderizado de plantillas PHP. La principal dificultad fue organizar el enrutamiento y el sistema de herencia de forma que fuera extensible sin romper lo ya construido.

OBJETIVO 2. Diseñar una base de datos relacional bien estructurada

El esquema de base de datos se definió en un único archivo SQL que actúa como fuente para toda la instalación y puede ejecutarse en cualquier entorno limpio para reproducir la estructura exacta. El diseño se organizó en cinco bloques conceptuales con relaciones claras entre ellos:

- **Usuarios y autenticación**: gestión de cuentas, sesiones mediante tokens, recuperación de contraseña con tokens de un solo uso y expiración.
- **Organización**: departamentos, relación N:M entre usuarios y departamentos mediante tabla pivote, y permisos individuales por usuario.
- **Tickets**: la entidad central del sistema, con categorías, prioridad por niveles, etiquetas libres asociadas mediante tabla pivote, y borrado lógico.
- **Actividad de tickets**: comentarios (internos o visibles al usuario), historial de cambios de campo con valor anterior y nuevo, y archivos adjuntos con ruta de almacenamiento.
- **Sistema**: notificaciones por usuario, configuración global clave valor y registro de auditoría de todas las acciones relevantes.

OBJETIVO 3. CRUD completo

Se han implementado operaciones completas de creación, lectura, actualización y borrado para las entidades principales: usuarios, tickets, departamentos, comentarios y adjuntos. Cada entidad tiene su propio modelo con métodos específicos, todos utilizando PDO con consultas preparadas para evitar inyección SQL.

OBJETIVO 4. Implementar un sistema de autenticación seguro

El sistema de autenticación incluye: login con email y contraseña (hash bcrypt), recuperación de contraseña mediante token de un solo uso enviado por email, flujo de cambio de contraseña obligatorio en el primer login (en cuentas creadas por un administrador), protección CSRF en todos los formularios POST, y posibilidad para desactivar cuentas sin borrarlas.

OBJETIVO 5. Controlar el estado de los tickets y las reparaciones

Los tickets siguen un flujo de estados definido: Sin Abrir → Abierto → En Progreso → Resuelto → Cerrado. Cada cambio de estado queda registrado. El estado inicial es sin_abrir que indica que el ticket solo ha sido creado pero ningún agente lo ha abierto todavía por lo que está a espera.

OBJETIVO 6. Implementar un sistema de permisos por rol

Se diseñó un sistema de permisos con dos niveles: defaults por rol y overrides individuales por usuario. Los permisos disponibles cubren todas las operaciones sensibles.

OBJETIVO 7. Implementar el envío de correos electrónicos

Se utilizó PHPMailer con configuración SMTP mediante variables de entorno (phpdotenv + .env). El wrapper Mailer.php encapsula toda la lógica de envío. Se usa para la recuperación de contraseña y las notificaciones automáticas al cliente sobre cambios de estado en su ticket.

OBJETIVO 8. Rol cliente con vinculación automática por email

Se diseñó un tercer rol cliente con acceso de solo seguimiento. Al registrarse o autenticarse, el sistema vincula automáticamente todos los tickets cuyo email coincide con el del usuario recién registrado.

Metodología

Hemos seguido un enfoque iterativo e incremental: primero diseñamos la base de datos, luego construimos el núcleo del MVC (FrontController, AppController, View, SPDO), y a partir de ahí fuimos añadiendo módulos funcionales uno a uno.

Hemos usado Git para el control de versiones y hemos ido ajustando el diseño según aparecían nuevas necesidades. Por ejemplo, se descartó la integración con WhatsApp Business API para las notificaciones por la complejidad del proceso de verificación de la cuenta y los costes asociados, además de la falta de tiempo. Aunque al final el email ha resultado ser suficiente y más fácil de implementar.

6. DISEÑO Y RESOLUCION DEL TRABAJO REALIZADO

6.1 Arquitectura MVC

La arquitectura del proyecto sigue el patrón ModeloVistaControlador implementado íntegramente en PHP sin frameworks externos. El punto de entrada único es index.php, que instancia el FrontController.

Flujo de una petición:

```
index.php
  └─> FrontController::dispatch()
      └─> Lee ?controller= y ?action= de la URL
      └─> Instancia el controlador correspondiente
      └─> Llama al método (acción)
          └─> Controlador usa Modelos (PDO) para acceder a datos
          └─> Controlador llama a View::render() con los datos
              └─> View incluye la plantilla PHP correspondiente
```

Componentes del núcleo (libs/):

Componente	Función
FrontController	Enruta por ?controller=&action=, instancia el controlador correcto
AppController	Clase base: sesión, redirecciones, renderizado, comprobación de login
View	Renderiza las plantillas PHP pasando variables
SPDO	Singleton de conexión PDO a MariaDB
Config	Lee variables de entorno desde .env vía phpdotenv
Csrf	Genera y valida tokens CSRF en formularios
Mailer	Wrapper de PHPMailer para el envío de emails

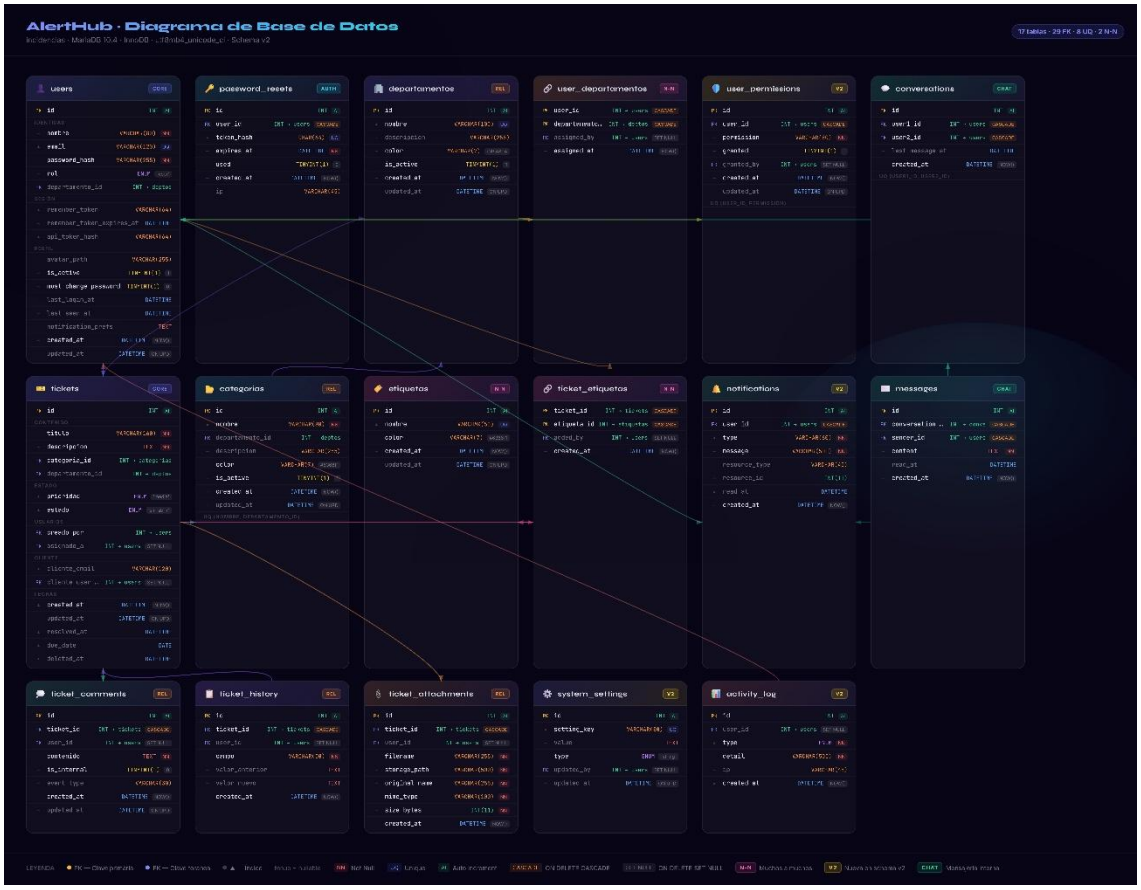
setup.php	carga autoload.php, inicia sesión, carga .env

6.2 Base de datos

La base de datos está diseñada en MariaDB 10.4 siguiendo principios de normalización. El esquema completo puede ejecutarse en cualquier instalación limpia para obtener la estructura exacta.

Entidades principales y sus relaciones:

- **users:** usuarios del sistema.
- **tickets:** incidencias. Relacionadas con el usuario creador, el asignado, la categoría y el departamento. Estado inicial sin_abrir.
- **categorías:** clasificación de los tickets.
- **etiquetas + ticket_etiquetas**: etiquetas libres asociadas a tickets mediante una tabla pivote.
- **ticket_comments:** comentarios en un ticket, pueden ser internos o visibles al cliente.
- **ticket_history:** registro automático de cada cambio de estado.
- **ticket_attachments:** archivos adjuntos con ruta de almacenamiento en uploads/.
- **departamentos** + **user_departamentos:** relación N:M usuarios y departamentos.
- **user_permissions:** overrides individuales de permisos sobre los defaults del rol.
- **notifications:** notificaciones de campana para cada usuario.
- **conversations + messages**: mensajería interna entre usuarios. conversations registra los usuarios y messages almacena el contenido.
- **activity_log:** auditoría de todas las acciones relevantes del sistema.
- **system_settings:** configuración global clave valor.
- **password_resets:** tokens de recuperación.



6.3 Sistema de autenticación y seguridad

El módulo de autenticación (AuthController + UsuarioModel + PasswordResetModel) cubre los siguientes flujos:

Login:

El usuario introduce email y contraseña. Se verifica el hash bcrypt con password_verify(). Si la cuenta tiene must_change_password = 1 (creada por un admin), se redirige al formulario de cambio de contraseña antes de dar acceso al dashboard.

Recuperación de contraseña:

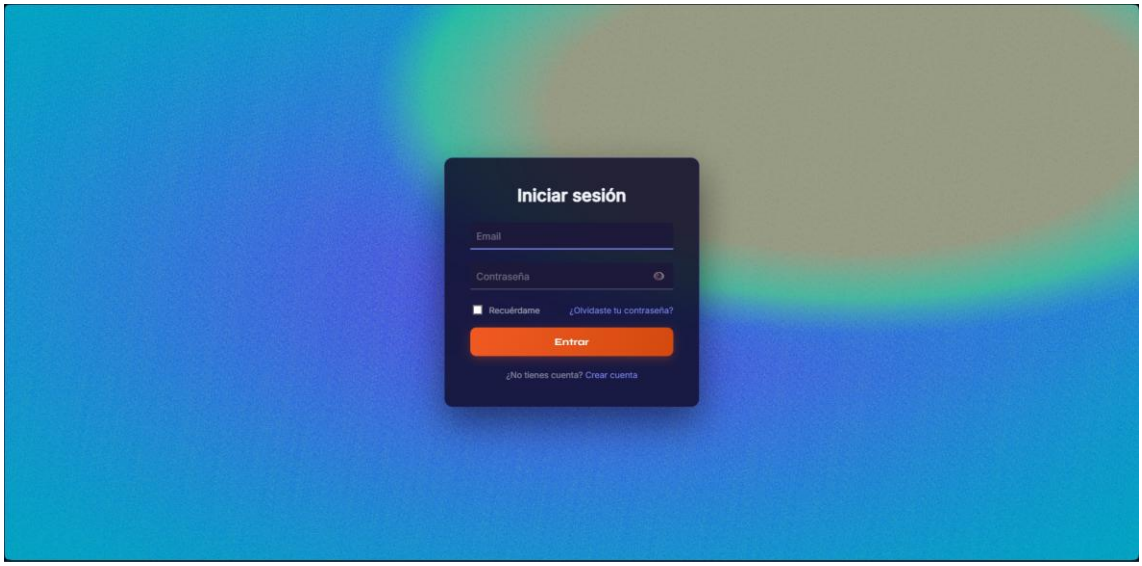
El usuario solicita un reset desde forgotView.php. Se genera un token único (bin2hex(random_bytes(32))), se almacena con expiración en password_resets, y se envía por email un enlace con ese token. Al acceder al enlace, resetView.php valida el token y permite establecer una nueva contraseña.

Protección CSRF:

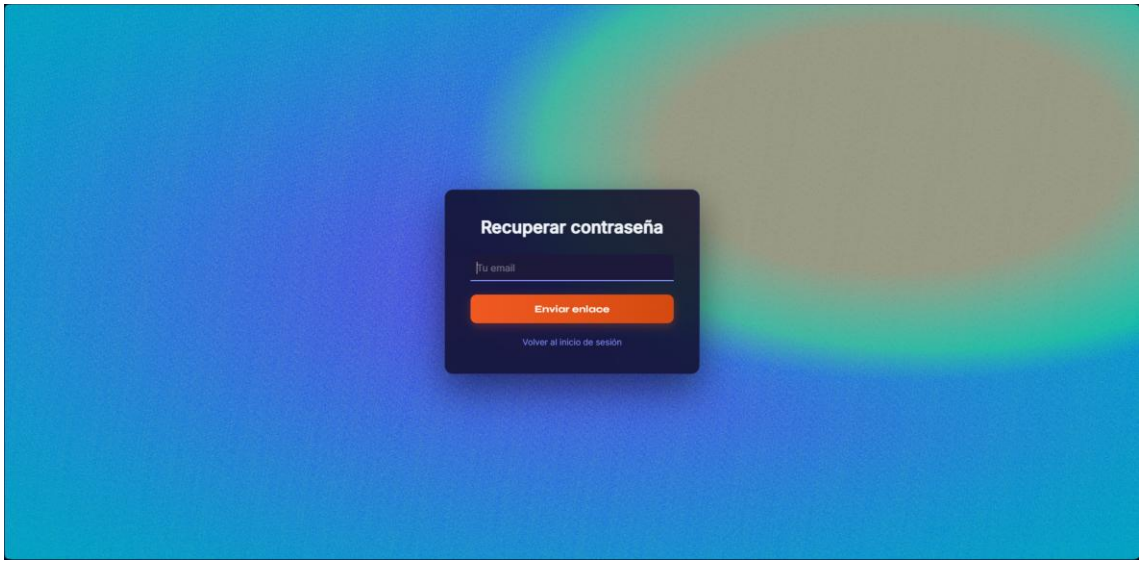
Todos los formularios POST incluyen un campo oculto generado por Csrf::field(). El controlador valida el token con Csrf::verify() antes de procesar cualquier dato. Si la validación falla, la petición es rechazada.

Variables de entorno:

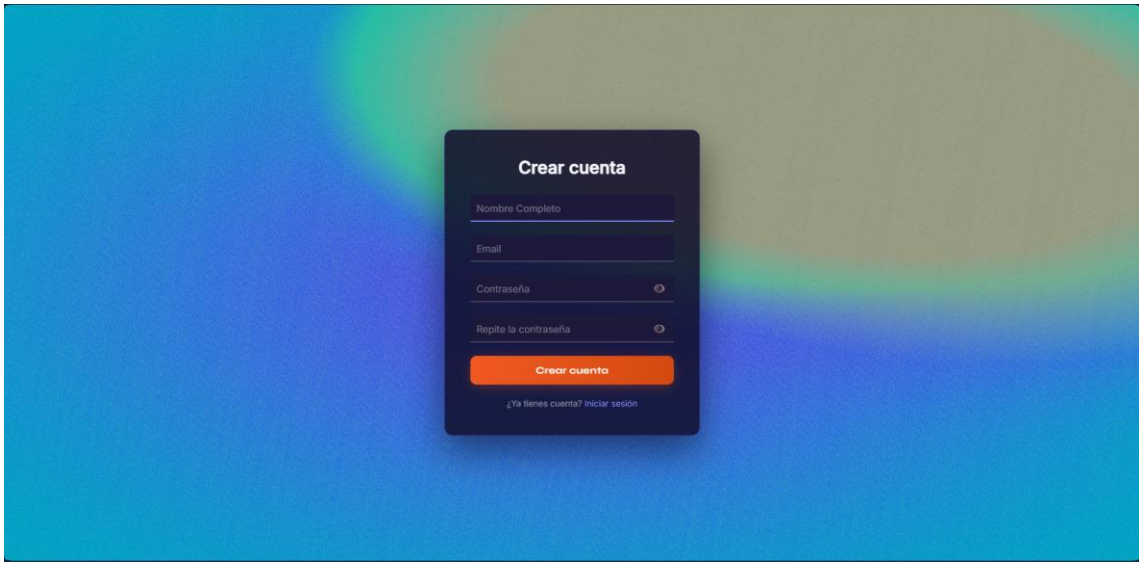
Las credenciales de base de datos y SMTP nunca están en el código fuente. Se leen desde el archivo. env mediante phpdotenv.



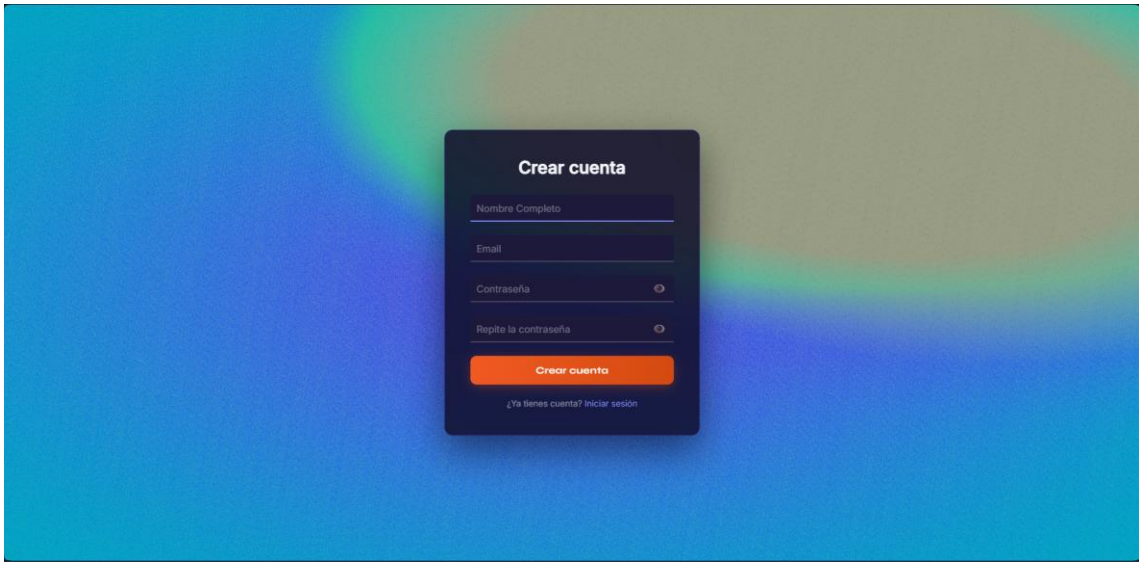
login de la app



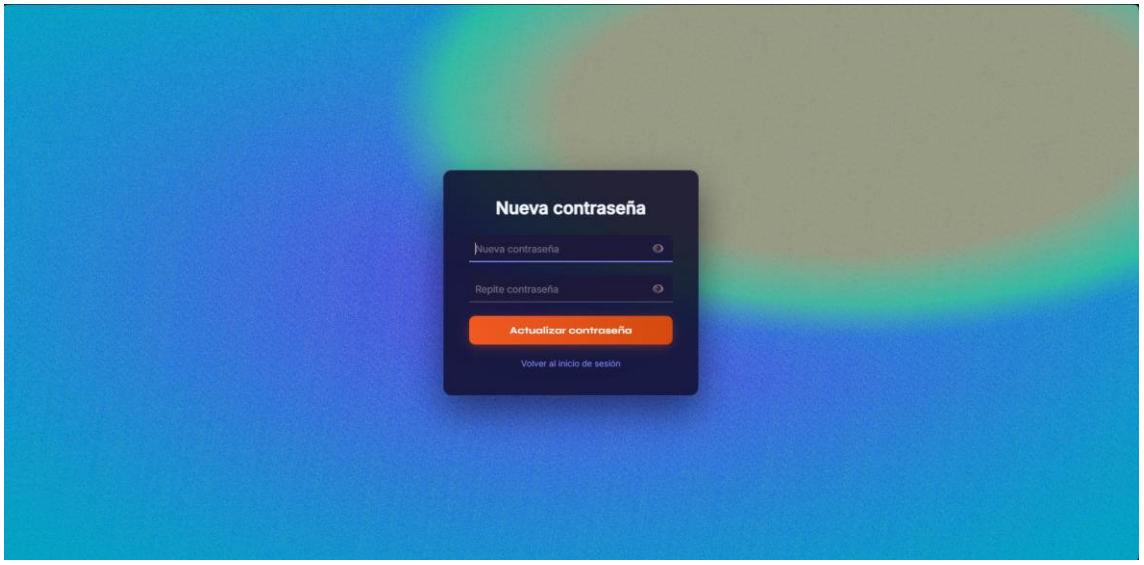
recuperar contraseña



crear cuenta



crear cuenta



cambiar contraseña

6.4 Roles de usuario

El sistema tiene tres roles:

- *admin:* acceso completo. Ve todos los tickets del sistema, puede crear tickets en nombre de otro usuario, reasignar, cambiar estado, editar y borrar. Tiene acceso al panel de configuración. Los admins ven los permisos individuales deshabilitados en el panel porque sus permisos no se pueden restringir.
- *user (agente):* ve los tickets de sus departamentos asignados, y los que creó o tiene asignados directamente. Puede comentar, cambiar estado y reasignar según sus permisos individuales.
- *cliente:* rol de solo seguimiento. Ve únicamente los tickets donde coincide con su ID. No puede crear tickets, comentar, cambiar estado ni ejecutar ninguna acción de gestión. La sidebar es mínima solo Mis Tickets y Mi Perfil. El registro público crea usuarios con este rol.

6.5 Gestión de tickets

Un ticket representa una incidencia reportada y gestionada por un agente de soporte.

Estados: Sin Abrir → Abierto → En Progreso → Resuelto → Cerrado. Cada transición queda en ticket_history. El estado sin_abrir indica que el ticket fue creado pero ningún agente lo ha abierto. Al acceder al detalle de un ticket en ese estado, el sistema lo pasa automáticamente a abierta y registra el evento en el timeline.

Creación: el formulario permite seleccionar departamento, categoría filtrada por el departamento elegido, prioridad, descripción y adjuntos. Se puede marcar un ticket como "ticket de cliente externo" e introducir el email del cliente, que se guarda en cliente_email. Si existe una cuenta con ese email, se rellena automáticamente. Los agentes ven solo los departamentos que tienen asignados en cambio los admins ven todos.

Auto-asignación (round-robin): si el modo de asignacion es round_robin en configuración, al crear un ticket se asigna automáticamente al agente del departamento con menos tickets activos. Si está en manual, queda sin asignar.Esta opción permite incluir admins en la rotación o quitarlos de la misma.

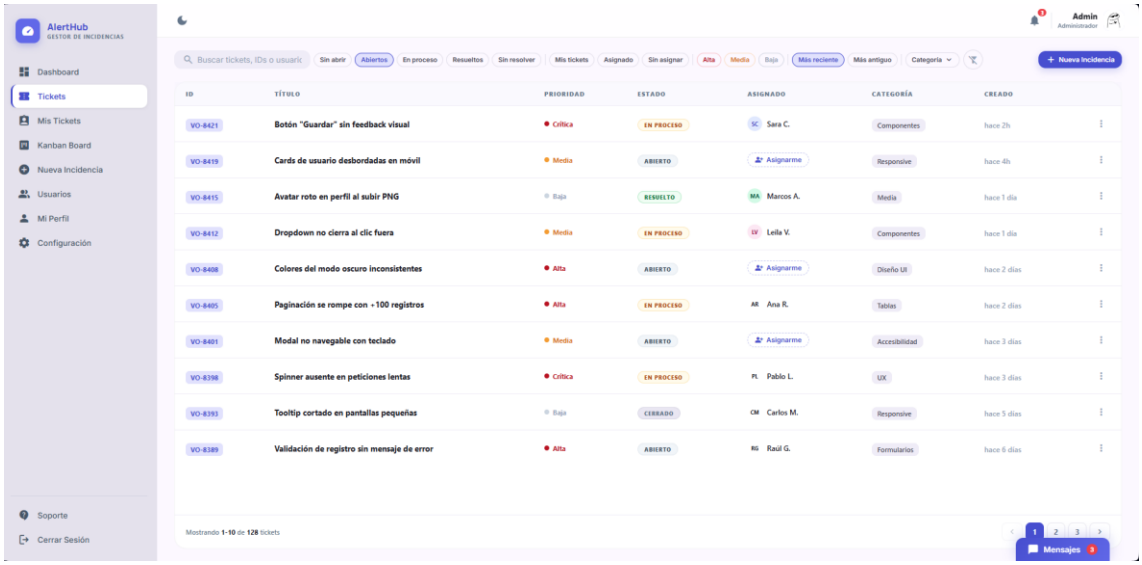
Edición: permite a los admins y agentes de soporte dueños del ticket editar título, descripción, categoría, prioridad, estado, asignado, cliente externo y fecha límite. Cada campo modificado genera una entrada en el historial del ticket con valor anterior y nuevo.

Archivos adjuntos: se almacenan en uploads/ con nombre aleatorio para evitar conflictos. El nombre original se guarda.

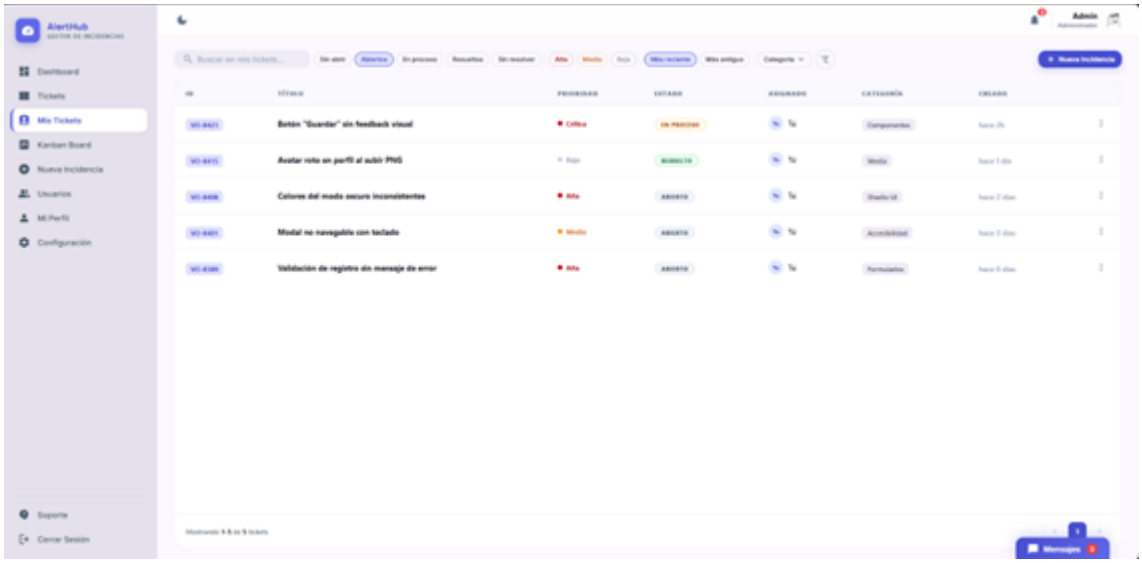
Visibilidad según rol: se genera la cláusula SQL correcta según el rol. Para clientes, filtra por el rol. Para agentes, filtra por departamentos asignados y los tickets propios. Para admins, sin filtro.

Vistas del módulo:

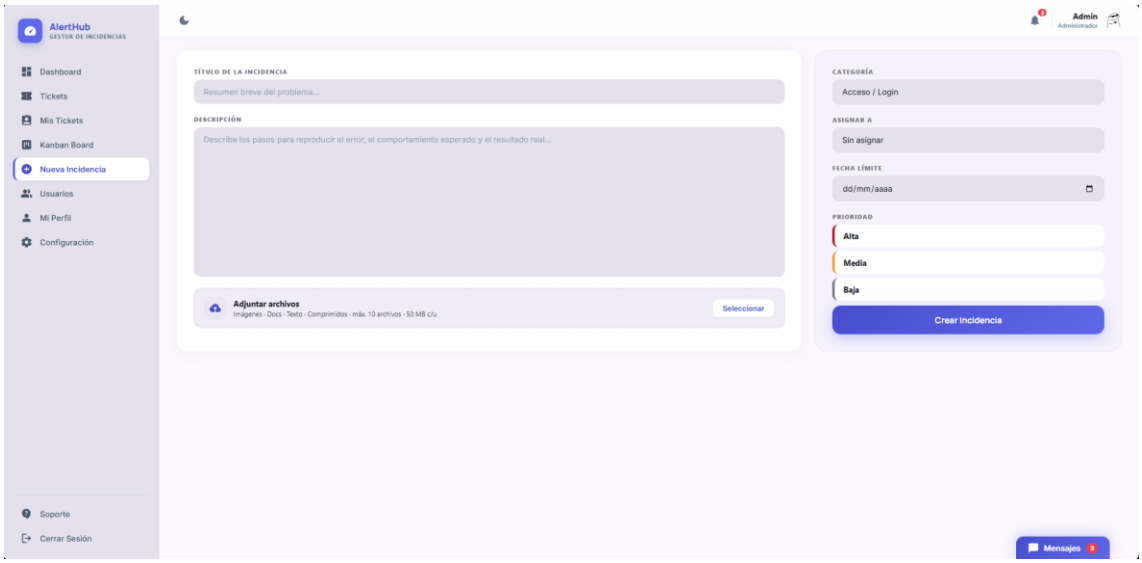
- ticketListView.php: listado con filtros por estado, departamento, prioridad y paginación. Los admins ven un dropdown extra de filtro por departamento.
- ticketDetailView.php: detalle con timeline de actividad, adjuntos, comentarios internos/externos, historial de cambios y metadatos.
- ticketNewView.php: formulario de creación.
- ticketEditView.php: formulario de edición (solo admins).
- misTicketsView.php: lista personal. Para clientes muestra lo mínimo necesario.



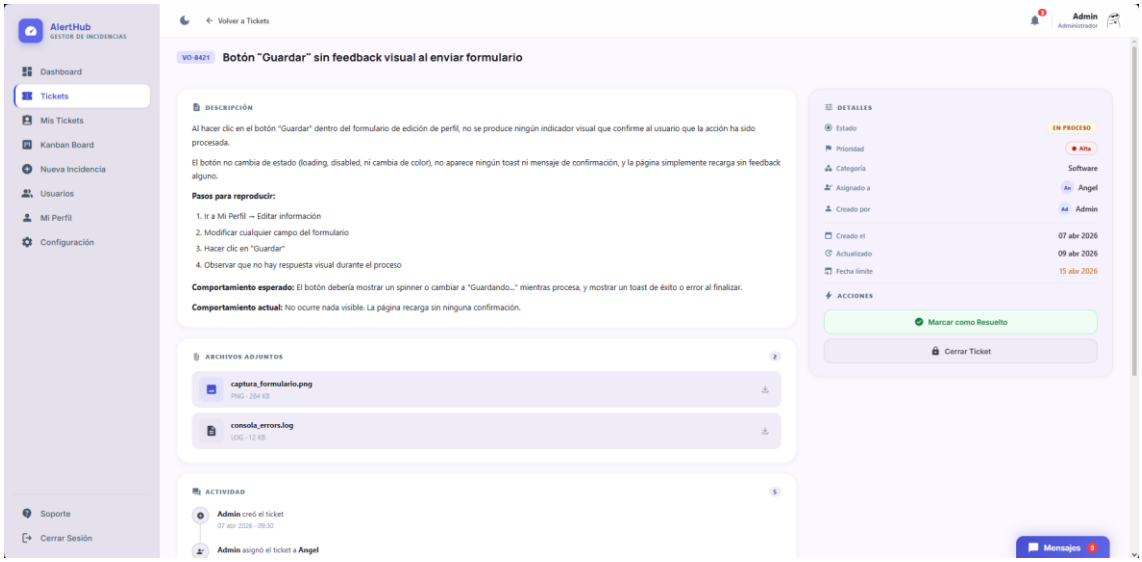
Lista de tickets general



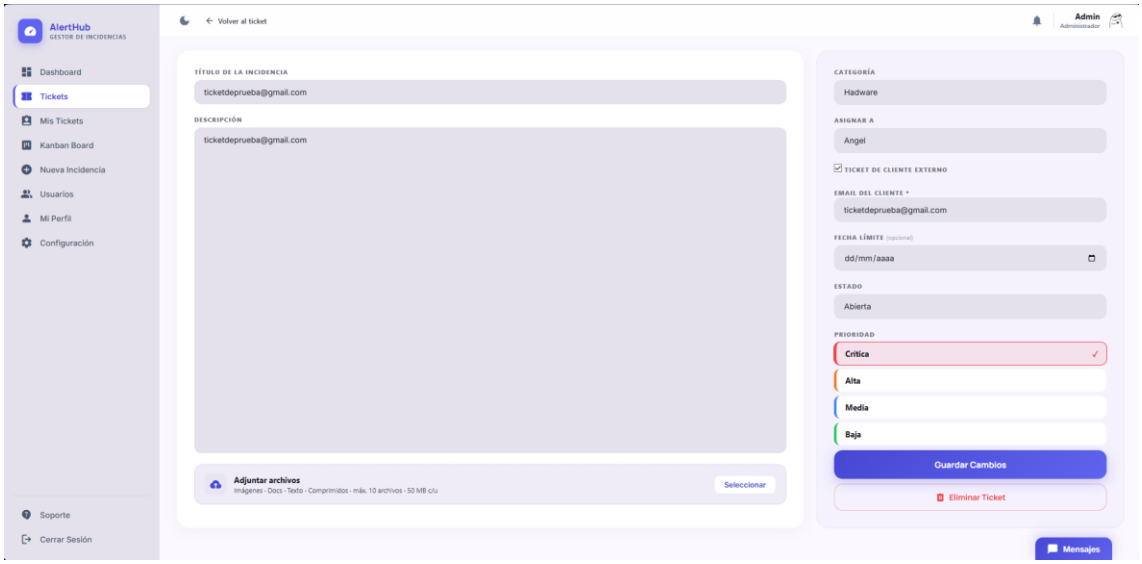
Lista de tickets privados de un usuario



Creación de tickets

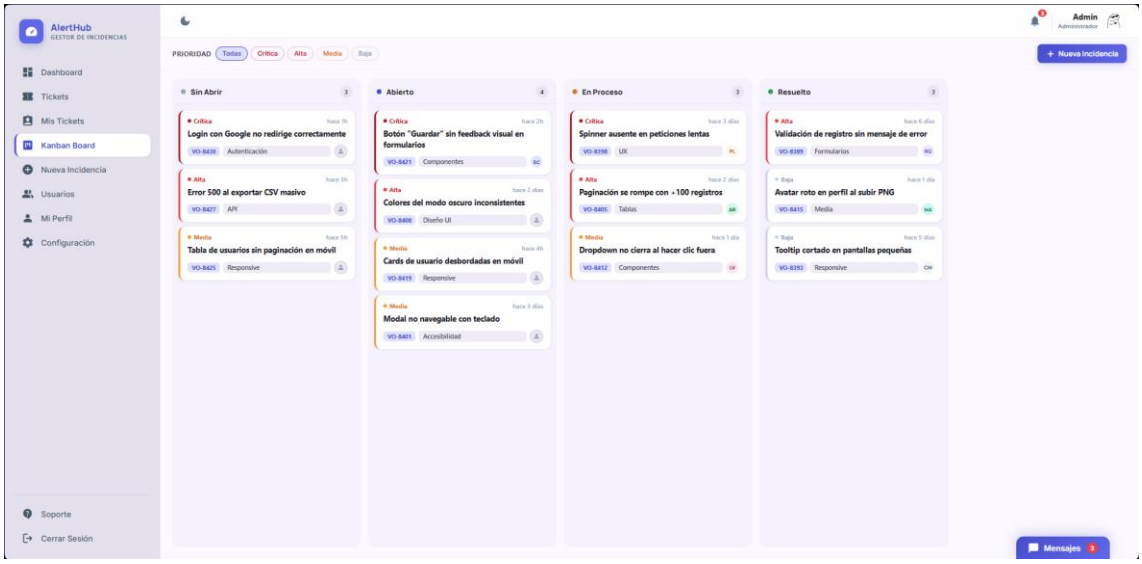


Detalles de un ticket



Edición de un ticket

Kanban: se genera un tablero donde las columnas son los estados. El acceso está controlado por el permiso acceso_kanban.



Kanban Board

6.6 Sistema de permisos granular

El permiso final de un usuario se resuelve en tres pasos:

1. Se consulta user_permissions en busca de un override individual para ese usuario y permiso.
2. Si existe el override, se aplica ese valor (se puede activar o desactivar un permiso que el rol por defecto no tendría).
3. Si no existe override, se aplica el valor por defecto del rol definido.

Permisos disponibles:

Permiso	Descripción
crear_tickets	Puede crear nuevas incidencias
ver_tickets_departamento	Ve solo tickets de sus departamentos asignados
ver_todos_tickets	Ve todos los tickets del sistema
comentar_tickets	Puede añadir comentarios
cambiar_estado_ajenos	Puede cambiar el estado de tickets de otros
reasignar_tickets	Puede reasignar tickets a otros agentes
cerrar_tickets_ajenos	Puede cerrar tickets que no creó
crear_en_nombre_de	Puede crear tickets en nombre de otro usuario
acceso_kanban	Acceso al tablero kanban
acceso_configuracion	Acceso al panel de configuración del sistema

El rol cliente tiene todos los permisos en false lo único que puede hacer es consultar sus tickets asignados.

6.7 Panel de Configuración

Solo accesible a admins (requiere permiso acceso_configuracion). Esta vista tiene seis secciones principales:

Gestión de usuarios: tabla con todos los usuarios del sistema. Acciones: activar/desactivar cuenta, cambiar rol, forzar cambio de contraseña en el próximo login, quitar avatar, y editar nombre/email. También permite crear nuevos usuarios con contraseña temporal (1234) que obliga a cambiar su contraseña en próximo login.

Departamentos de usuario: desde el panel de cada usuario se pueden asignar y quitar departamentos.

Permisos especiales: panel por usuario que escriben sus permisos tanto los defaults como los extras que pueda tener o quitarse.

Categorías: CRUD completo con activo/inactivo. Cada categoría tiene un conteo de tickets asociados a cada categoría. Cada categoría pertenece a un departamento.

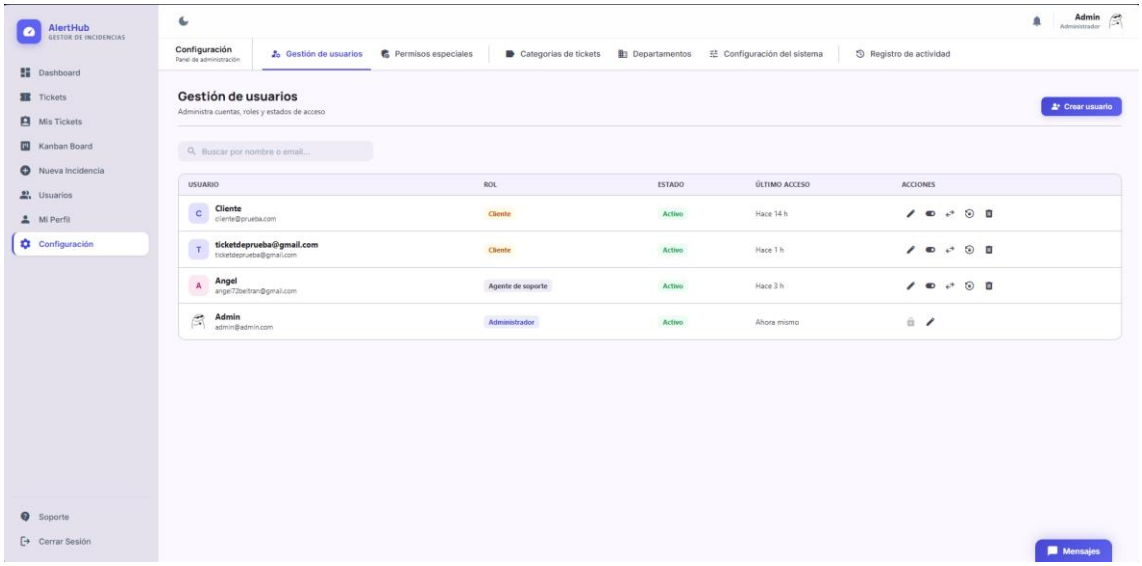
Departamentos: CRUD completo con conteo de miembros pertenecientes a un departamento.

Etiquetas: CRUD de etiquetas libres que los agentes pueden asociar a tickets.

Sistema: formulario que guarda todos los ajustes. Configuración disponible:

Clave	**Descripción**
registro_abierto	Permite o bloquea el registro público de nuevos usuarios
max_tickets_por_usuario	Máximo de tickets activos por usuario
inactividad_minutos	Minutos sin actividad para considerar usuario inactivo
zona_horaria	Zona horaria del sistema
formato_fecha	Formato de visualización de fechas
modo_asignacion	manual o round_robin
incluir_admins_rotacion	Si los admins entran en la rotación round-robin
modo_mantenimiento	Activa/desactiva el modo mantenimiento

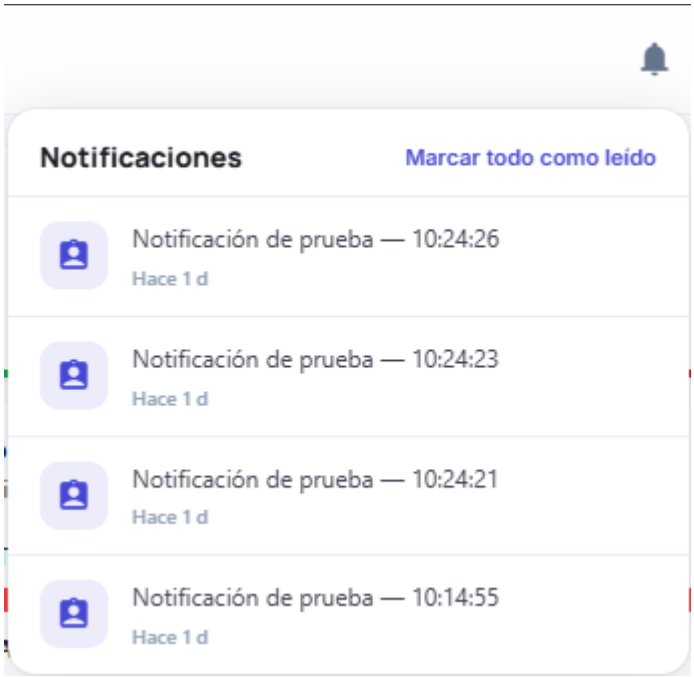
Registro de actividad: Registro completo de todo lo que ocurre en la página web, cada acción se registra y se muestra. Incluye búsqueda adicional en cliente.



panel de configuración

6.8 Notificaciones

La campana en la cabecera del dashboard muestra el conteo de notificaciones no leídas. Cada notificación se puede marcar como leída o marcar todas como leídas. Había varios tipos de notificaciones: por asignar un ticket, por comentar en un ticket tuyo, si cambia el estado de un ticket, si crean un ticket que se te asigna y si el ticket vence de fecha límite.

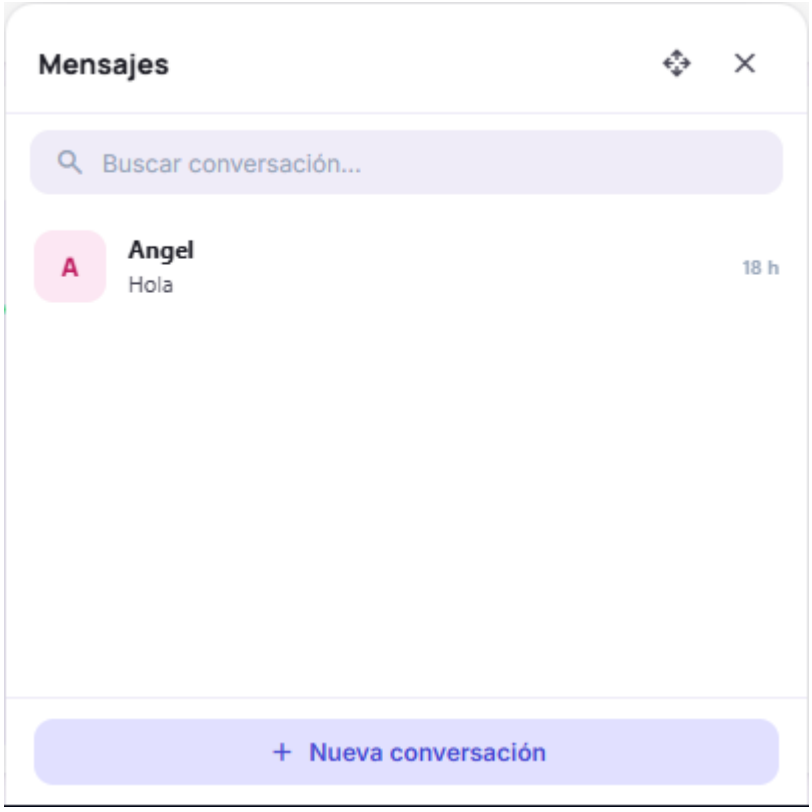


Cada usuario en su perfil tiene la capacidad de tener unas preferencias de notificación que se guardan como JSON.

6.9 Mensajería Interna

Las tablas conversations y messages gestionan la mensajería directa entre usuarios. Se abre o se recupera una conversación existente entre dos usuarios y se devuelve la lista de usuarios disponibles: todos para admins, solo los del mismo departamento para agentes.

La interfaz de mensajes se carga con messages.js sin recargar la página. El panel de mensajes está accesible desde cualquier vista del dashboard.

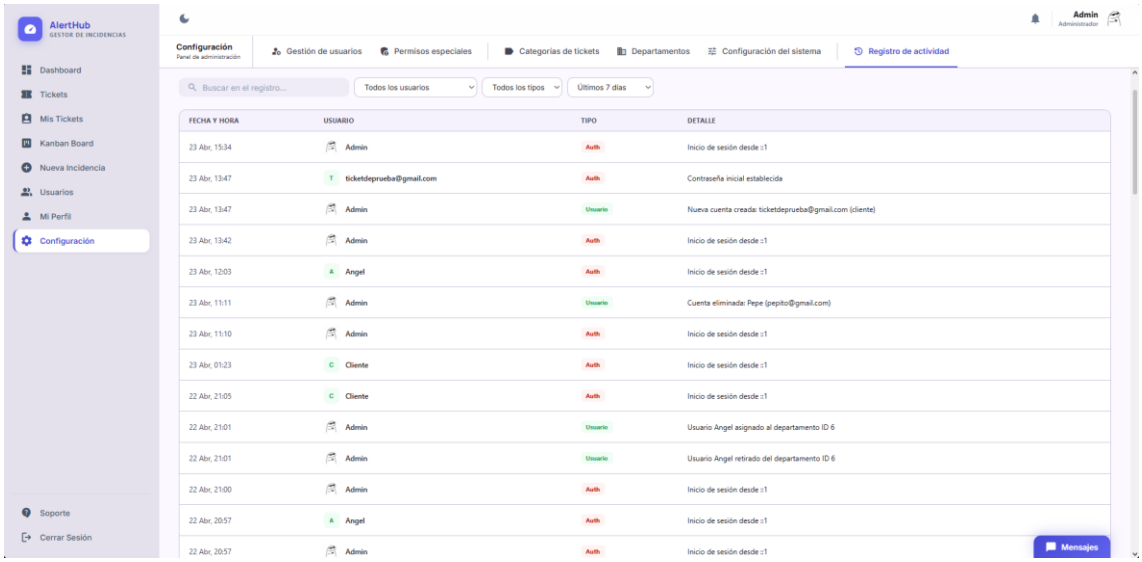


6.10 Presencia y actividad

Hay un endpoint que actualiza la última conexión del usuario en sesión y devuelve un JSON con la hora, presence.js hace ping a este endpoint periódicamente desde el frontend, lo que permite saber qué usuarios están activos en tiempo real.

6.11 Auditoría

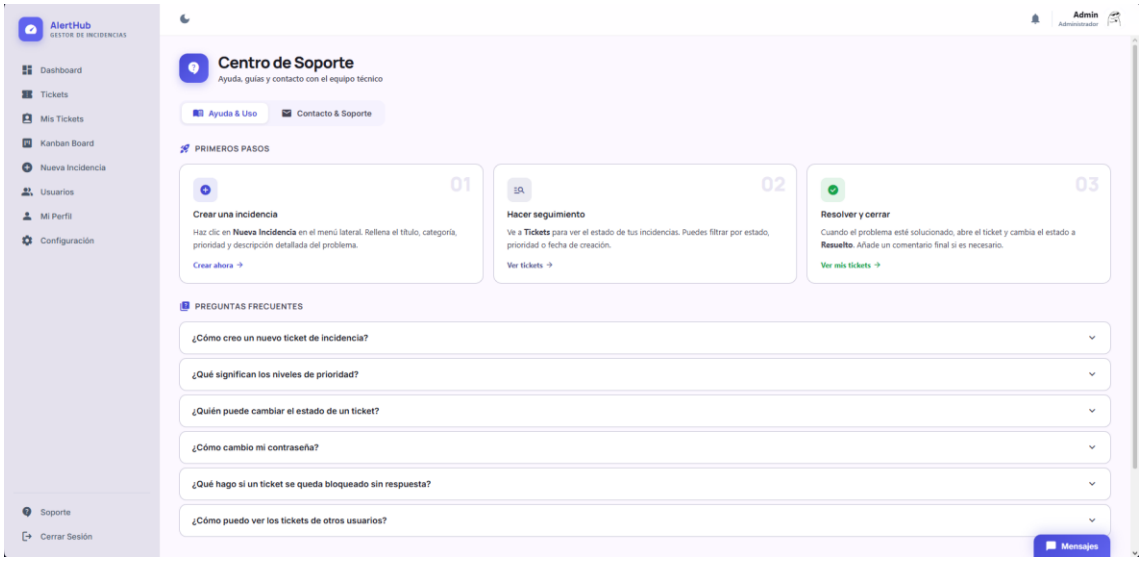
Toda acción relevante se registra: login, cambio de contraseña, modificaciones de permisos, cambios en tickets y cambios en la configuración del sistema. El admin puede filtrar desde el panel de configuración.

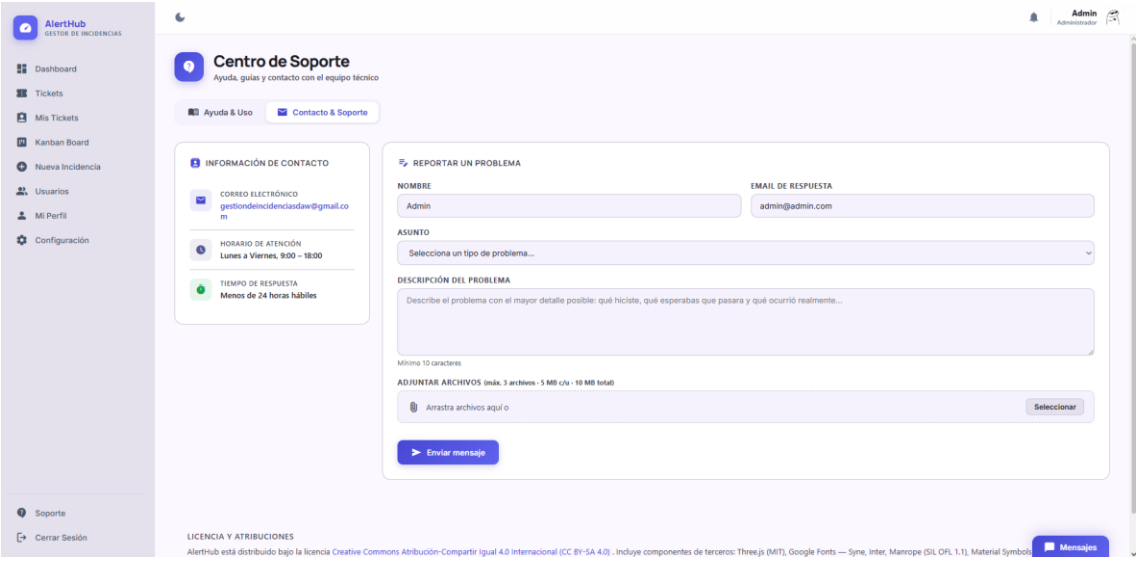


registro de auditoria

6.12 Vista de soporte

La vista incluye un apartado de preguntas frecuentes y explicación rápida sobre el uso de la web y un formulario de contacto con validación y soporte para adjuntos múltiples: máximo 3 archivos, 5 MB por archivo, 10 MB en total. La whitelist de extensiones permite: .pdf, .docx, .xlsx, .txt, .json, .xml y similares. El envío se procesa con PHPMailer.



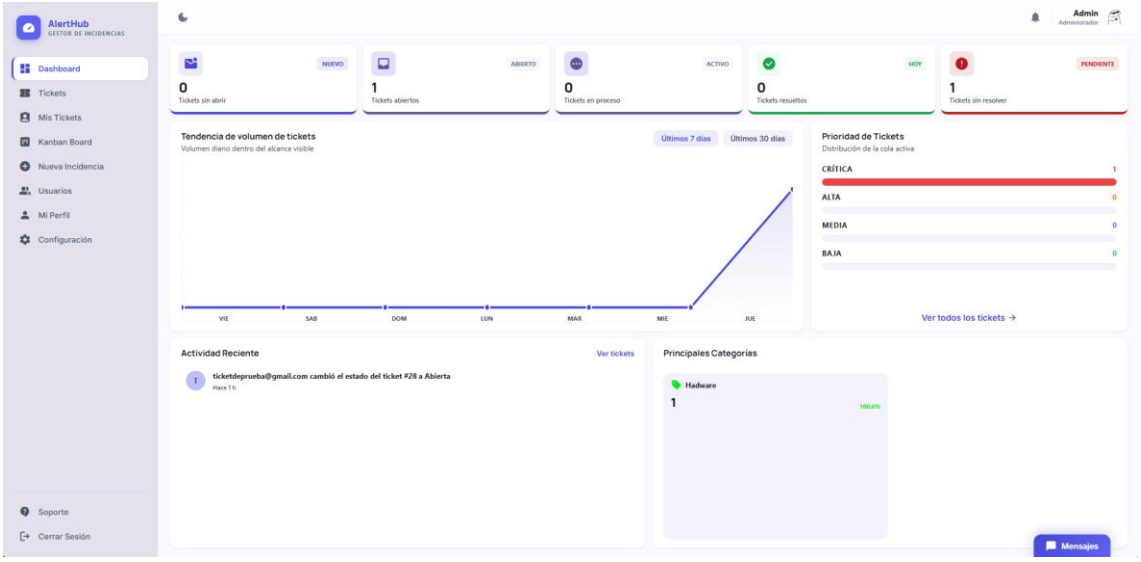


6.13 Dashboard

El dashboard recopila métricas en tiempo real entre ellas las siguientes:

- Conteo de tickets por estado
- Tendencia de volumen en los últimos 7 y 30 días
- Distribución por prioridad de tickets activos
- Últimas acciones del sistema
- Categorías con más tickets

Los usuarios con rol cliente son redirigidos a misTickets en lugar del dashboard, ya que no tienen visibilidad sobre las métricas globales.



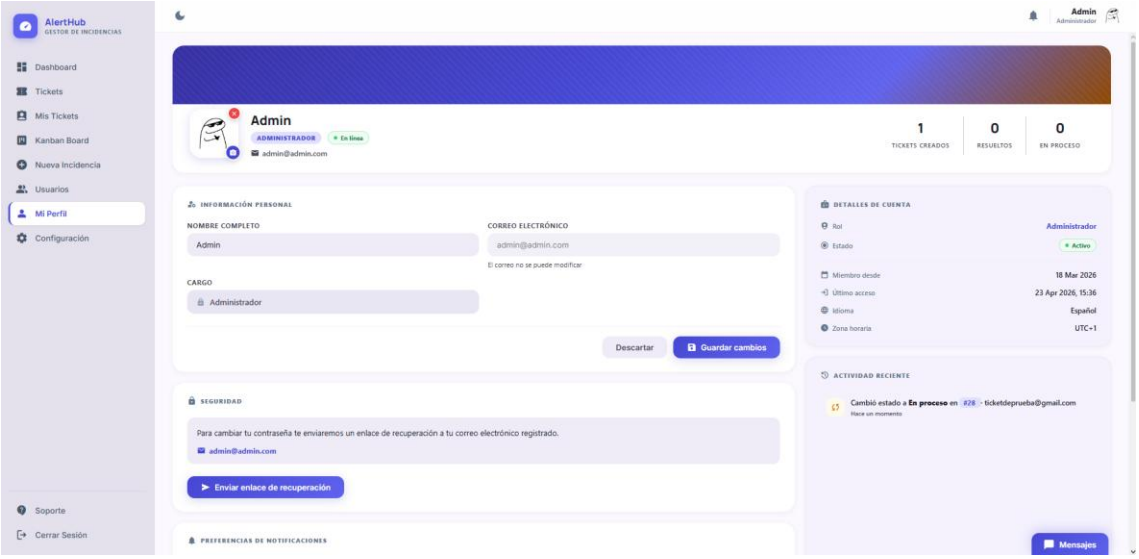
6.14 Interfaz y experiencia de usuario

Las cuatro vistas de autenticación (login, registro, recuperación y reset) comparten un diseño diferenciado: Three.js r128 renderiza un gradiente líquido animado con WebGL como fondo, implementado en liquid-gradient.js. Las tipografías son de Google Fonts: Syne para títulos e Inter para el cuerpo de texto.

El resto de la aplicación usa CSS propio sin frameworks externos, organizado en hojas de estilo por módulo. La fuente principal del dashboard es Manrope (Google Fonts). Los iconos son Material Symbols Outlined, cargados desde Google Fonts.

La aplicación tiene soporte para tema claro y oscuro. theme.js gestiona el switch y almacena la preferencia en localStorage. La navegación lateral se controla mediante sidebar.js. El panel de notificaciones se gestiona con notifications.js y el de mensajes con messages.js.

El módulo de perfil permite al usuario cambiar su avatar: subida de imagen con validación de tipo (JPG, PNG, WEBP) y tamaño (máximo 2 MB). También accede al formulario de preferencias de notificación y al botón de envío de enlace de recuperación de contraseña.



mi perfil

6.15 Tecnologías utilizadas

- PHP 8.2
- MariaDB 10.4
- Apache / XAMPP (desarrollo local)
- CSS3 propio por módulo, sin framework externo
- Three.js r128 (gradiente WebGL en vistas de auth)
- Google Fonts: Syne + Inter (auth) y Manrope + Inter (dashboard)
- Material Symbols Outlined (iconos del dashboard)
- PHPMailer (correos SMTP)
- phpdotenv (variables de entorno desde .env)
- Composer (gestión de dependencias PHP)
- Git (control de versiones)

6.16 Dificultades encontradas

Framework MVC propio: encadenar el enrutamiento, la instanciación dinámica de controladores y el sistema de herencia fue la primera dificultad grande. Tuvimos que estudiar cómo Laravel resuelve esto internamente para implementar nuestra versión simplificada.

Sistema de permisos: el primer diseño basado solo en roles no era suficientemente flexible. Rediseñamos la tabla y adaptamos todas las comprobaciones ya existentes en los controladores para admitir overrides individuales.

Flujo must_change_password: implementar la redirección al cambio de contraseña sin causar bucles infinitos requirió añadir lógica de intercepción en el AppController antes de ejecutar cualquier controlador destino. Cualquier ruta protegida que un usuario con must_change_password = 1 intentara acceder tenía que ser interceptada y redirigida.

Rol cliente y vinculación por email: definir qué ve el cliente y cuándo se ejecuta la vinculación (al registrarse, al hacer login, y en cada acceso a misTickets) requirió varias iteraciones para cubrir todos los casos sin duplicar entradas ni crear condiciones.

Sesión y CSRF: garantizar que los tokens CSRF fueran correctamente regenerados y validados en todos los formularios, incluidos los de subida de archivos, requirió revisar el proyecto entero.

Auto-asignación round-robin: implementar la asignación por el agente con menos tickets activos del departamento correcto, respetando la opción de incluir admins en rotación, requirió varias pruebas.

7. MANUAL DE INSTALACIÓN Y DESPLIEGUE

7.1 Requisitos Mínimos del Sistema

Sistema Operativo:

Windows, Linux o macOS

Servidor Web:

- Apache (recomendado XAMPP en entorno local)
- MySQL 8.0 o MariaDB 10.4
- GIT

PHP 8.2 o superior con las siguientes extensiones habilitadas:

- pdo y pdo_mysql: conexión a base de datos mediante PDO.
- mbstring: manejo de cadenas multibyte.
- openssl: cifrado y generación de tokens seguros.
- fileinfo: validación de tipo MIME en subida de archivos.
- json: procesamiento de datos JSON.

Dependencias de la aplicación (instaladas vía Composer):

- PHPMailer: envío de correos mediante SMTP
- Phpdotenv: carga de variables de entorno desde el archivo .env

Navegador Web:

Chrome, Firefox, Edge o Safari (versión moderna)

Espacio y Recursos:

- **Espacio en disco**: 3GB mínimo + espacio para archivos adjuntos
- **Memoria RAM**: 2 GB mínimo recomendado
- **Procesador**: 2 núcleos mínimo

7.2 Proceso de Instalación Paso a Paso

PASO 1: Clonar o descargar el repositorio

Sitúate en la carpeta raíz de tu servidor web y clona el proyecto:

En XAMPP (Windows):

cd C:/xampp/htdocs

En Linux/macOS:

cd /var/www/html

git clone https://github.com/angelbeltran72/TFG

PASO 2: Instalar dependencias

Ejecuta el siguiente comando en la carpeta del proyecto para instalar PHPMailer, phpdotenv y otras dependencias:

composer install

PASO 3: Configurar Variables de Entorno

Copia el archivo `.env.example` y renómbralo a `.env`.

Abre el archivo y rellena los valores:

URL base de la aplicación

APP_URL=http://localhost/TFG

COOKIE_SECURE=false

Base de datos

DB_HOST=localhost

DB_NAME=incidencias

DB_USER=root

DB_PASS=tu_contraseña

DB_CHARSET=utf8mb4

Correo SMTP (ejemplo con Gmail)

MAIL_HOST=smtp.gmail.com

MAIL_PORT=587

MAIL_USERNAME=tu_correo@gmail.com

MAIL_PASSWORD=tu_app_password

MAIL_FROM_ADDRESS=tu_correo@gmail.com

MAIL_FROM_NAME="Gestor de Incidencias"

Para usar Gmail como servidor SMTP es necesario generar una contraseña de aplicación desde la configuración de seguridad de tu cuenta Google, no se usa la contraseña normal de Gmail.

PASO 4: Base de Datos

Crea el esquema de la base de datos utilizando el archivo `database/Base de Datos Completa.sql` en tu servidor MariaDB.

Línea de comandos

mysql -u root -p -e "CREATE DATABASE incidencias CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"

mysql -u root -p incidencias < "database/Base de Datos Completa.sql"

phpMyAdmin

1. Crea una base de datos llamada "incidencias" con cotejamiento utf8mb4_unicode_ci
2. Selecciónala y ve a la pestaña "Importar"
3. Sube el archivo database/Base de Datos Completa.sql

PASO 5: Verificar permisos de la carpeta de adjuntos

La carpeta uploads/ necesita permisos de escritura para que el servidor pueda guardar los archivos adjuntos:

En Linux/macOS:

chmod -R 775 uploads/

En Windows

Con XAMPP no suele ser necesario ningún ajuste adicional.

PASO 6: Acceder a la aplicación

Abre tu navegador y navega a:

http://localhost/TFG

La aplicación redirigirá automáticamente al formulario de inicio de sesión. Puedes registrar tu primera cuenta desde la pantalla de registro — el primer usuario que se registre deberá ser promovido a administrador directamente en la base de datos o a través del panel de configuración si ya existe un admin previo.

7.3 Guia de Usuario Final

7.3.1 Acceso e Inicio de Sesión

Registro y primer acceso al Sistema:

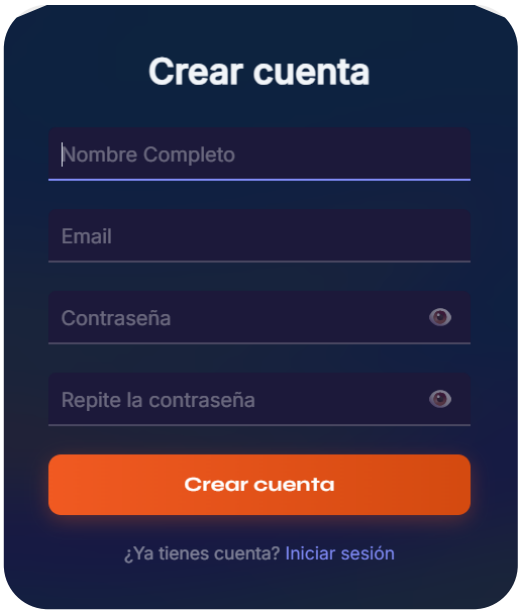
1. Navega a la URL configurada previamente.
2. Haz clic en Crear cuenta.
3. Rellena los campos requeridos y pulsa Crear cuenta.
4. Una vez registrado, haz clic en Iniciar Sesión e introduce tus credenciales.

El registro público crea cuentas con rol cliente por defecto. Si necesitas acceso como agente o administrador, tendrás que utilizar la cuenta de administrador para cambiar tu rol desde el panel de configuración.

usuario: admin@admin.com

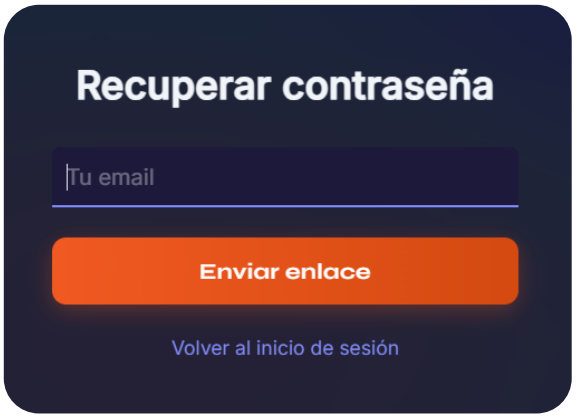
contraseña: admin123

Estos son credenciales de demo del entorno de pruebas los cuales deberás dar permisos de administrador a tu cuenta principal y eliminar esta antes de cualquier uso real.



Recuperación de Contraseña:

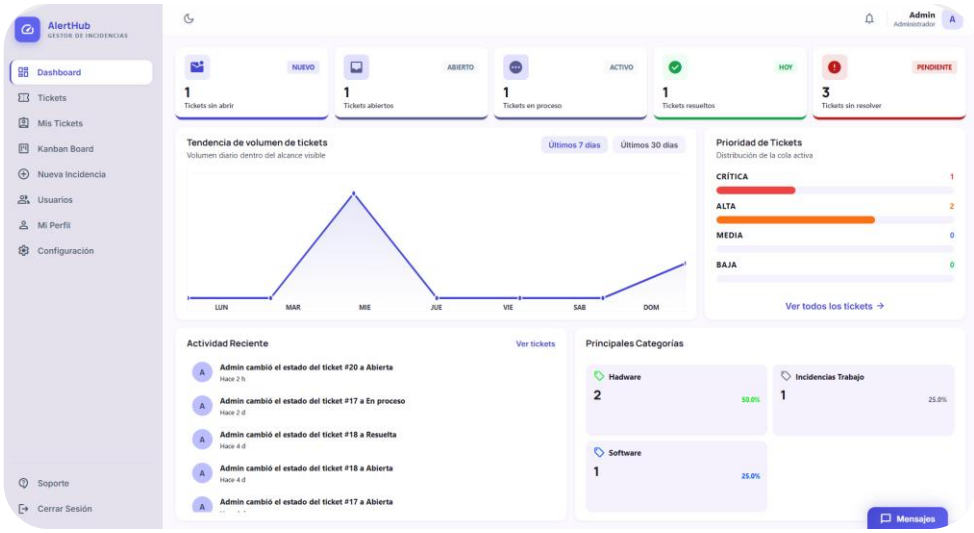
1. En la pantalla de login haz clic en "¿Olvidaste tu contraseña?".
2. Ingresar dirección de email registrada.
3. Revisar bandeja de entrada y seguir instrucciones del correo.
4. Establecer nueva contraseña segura.



7.3.2 Panel Principal y Navegación

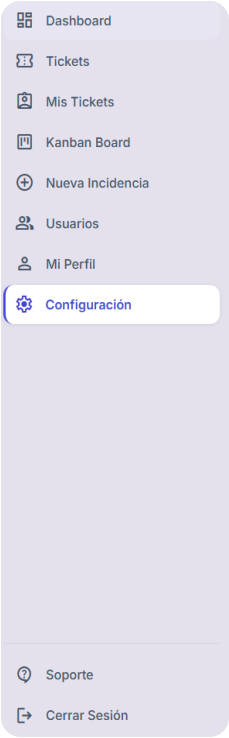
Dashboard de bienvenida:

- Resumen visual de tickets activos y su distribución por estado
- Volumen de incidencias de los últimos 7 y 30 días
- Distribución de tickets activos por prioridad
- Últimas acciones registradas en el sistema
- Categorías con más tickets abiertos



Menú Principal:

Opción	Descripción
Dashboard	Pantalla principal con métricas globales
Tickets	Listado completo de incidencias del sistema
Mis Tickets	Incidencias asignadas al usuario en sesión
Kanban Board	Vista visual de tickets por estado
Nueva Incidencia	Formulario de creación de ticket
Usuarios	Gestión de usuarios (requiere permisos)
Mi Perfil	Configuración personal del usuario
Configuración	Panel de administración del sistema (solo admin)
Soporte	Preguntas frecuentes y formulario de contacto
Cerrar sesión	Finaliza la sesión activa

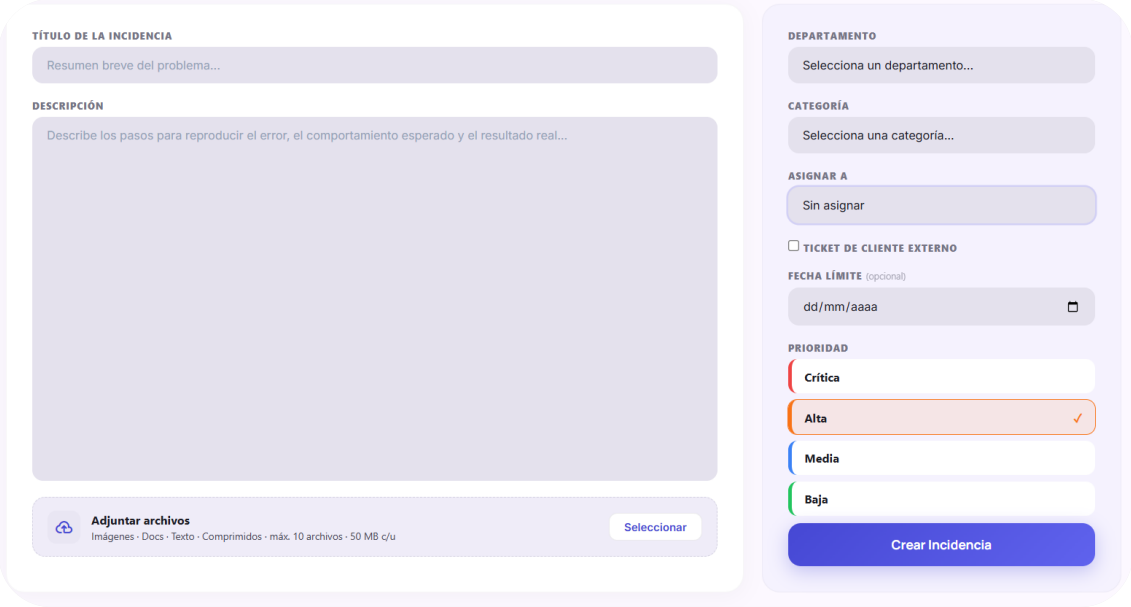


7.3.3 Gestión completa de incidencias

Crear Nueva Incidencia:

1. **Navegar** a "Nueva Incidencia".
2. **Describir detalladamente** el problema o solicitud introduciendo título y una descripción.
3. **Categorizar** incidencia (Departamento, Categoría).
4. **Asignar** usuario.
5. **Asignar Prioridad** según impacto:
   - **Baja**: Problema menor sin afectación operativa.
   - **Media**: Afectación parcial sin impacto crítico.
   - **Alta**: Afectación operativa significativa.
   - **Crítica**: Sistema completamente inoperativo.

6. **Adjuntar archivos** si es necesario (capturas, documentos).
7. Pulsa **Crear Incidencia** para guardar y crear la incidencia.



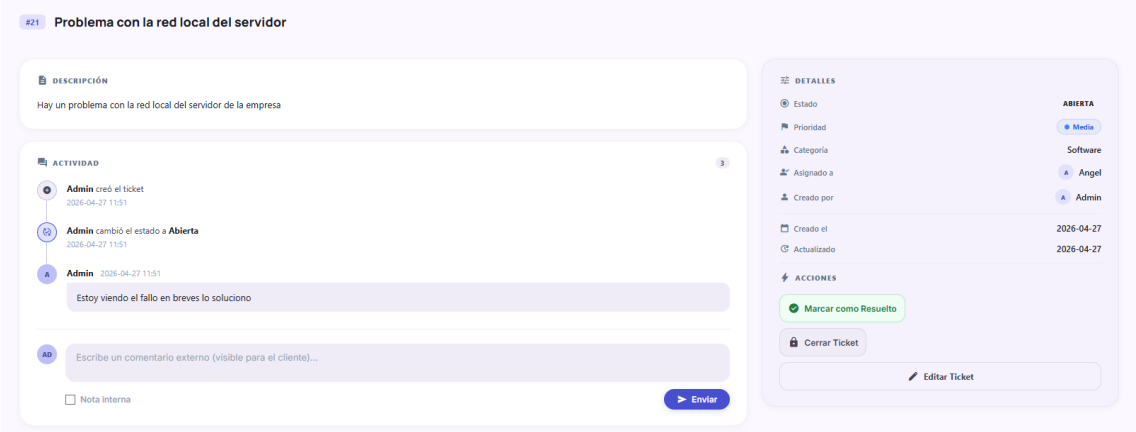
Flujo de Estados de Incidencia:

Sin Abrir→ Abierto → En Progreso → Resuelto → Cerrado

- Sin Abrir: el ticket ha sido creado pero ningún agente lo ha revisado aún.
- Abierta: un agente ha accedido al ticket (cambio automático).
- En Proceso: el agente está trabajando en la resolución.
- Resuelta: el problema ha sido solucionado, pendiente de confirmación.
- Cerrado: incidencia finalizada y archivada.

Seguimiento y Actualización:

- Agregar comentarios con cada avance significativo pueden marcarse como internos (solo visibles para agentes o admins) o externos (visibles también para el cliente si hay).
- Cambiar estado según el progreso de la resolución.
- Reasignar a otro técnico si es necesario.



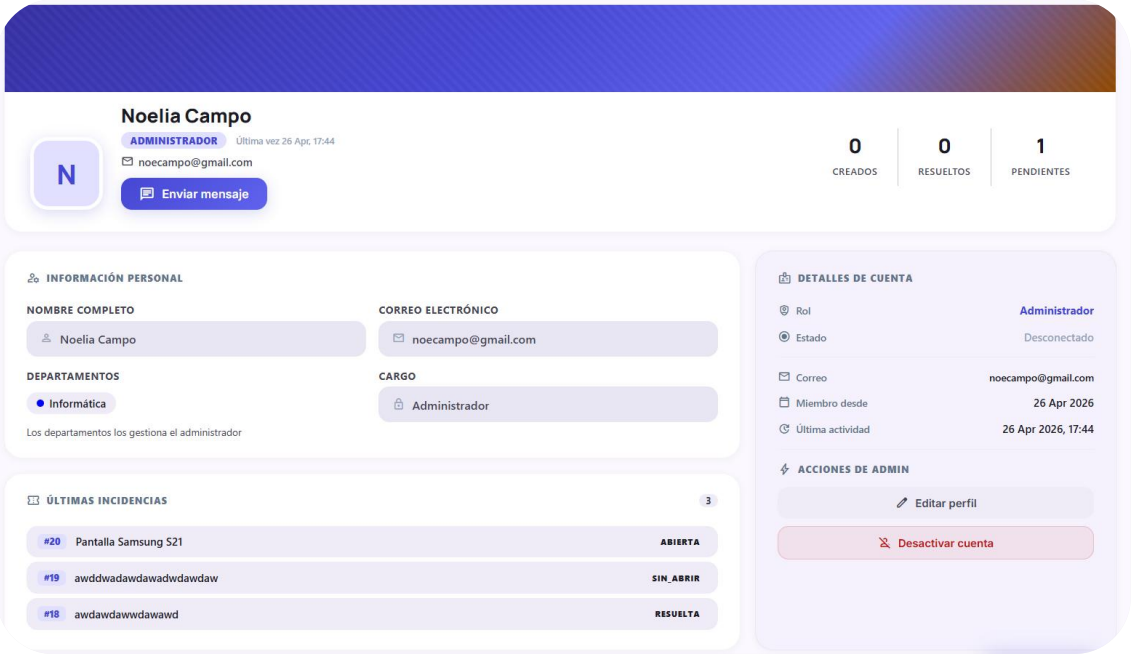
7.3.4 Gestión completa de Usuarios

Ver Usuarios:

1. Navegar a "Usuarios" en el menú principal.
2. Buscar cliente por nombre o email.
3. Ver información completa de contacto.
4. Revisar historial de incidencias anteriores.
5. Consultar información, Editar perfil o Desactivar Cuenta.

Acciones disponibles sobre cada usuario (siendo administrador):

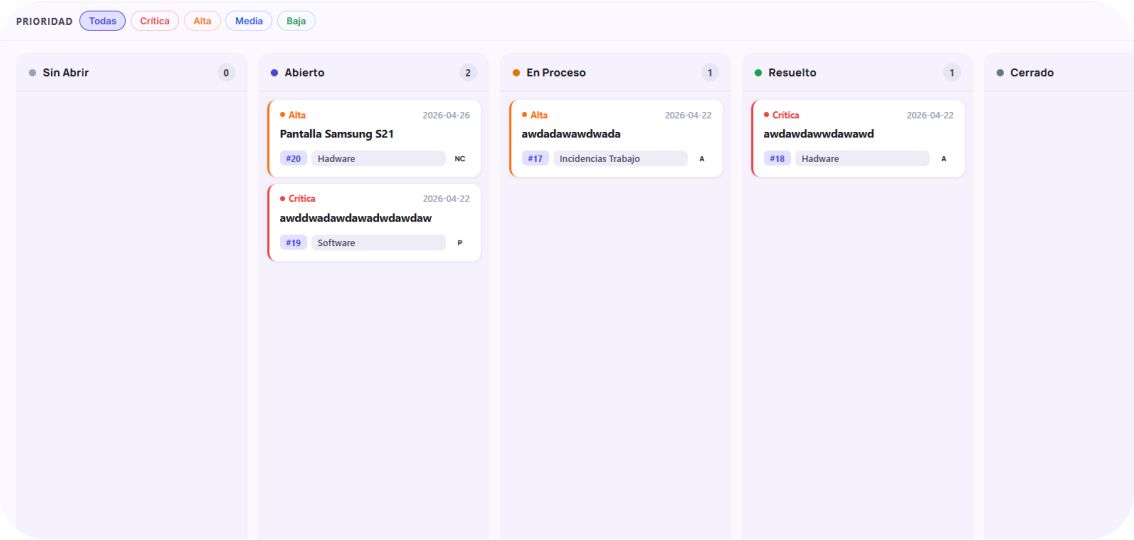
- Editar perfil: modifica nombre, email u otros datos del usuario.
- Desactivar cuenta: bloquea el acceso del usuario sin eliminar su historial.
- Cambiar rol: asigna el rol admin, agente o cliente.
- Forzar cambio de contraseña: el usuario deberá cambiar su contraseña en el próximo inicio de sesión.
- Eliminar cuenta: elimina el usuario del sistema de forma permanente.



7.3.5 Kanban Board

Visualización de tickets

- Dentro de cada columna los tickets se ordenan por prioridad.
- Haz clic sobre cualquier ticket para acceder a su vista de detalle completa.
- Permite cambiar el estado de los tickets arrastrándolo al estado deseado.

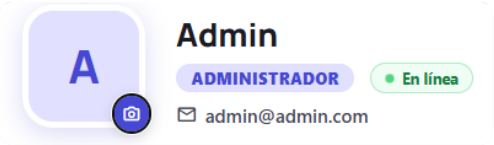


El acceso al Kanban Board está controlado por el permiso acceso_kanban. Los usuarios sin ese permiso no verán esta opción en el menú.

7.3.6 Mi Perfil

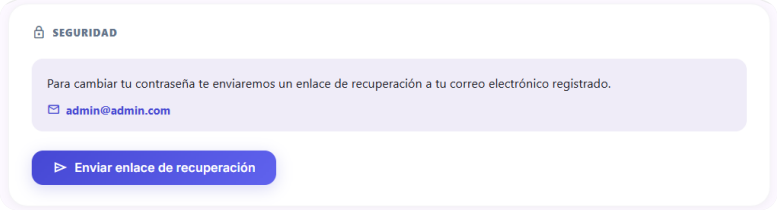
Actualizar avatar

Presiona en el botón que hay junto a tu foto de perfil para adjuntar una nueva imagen y selecciona una imagen (formatos admitidos: JPG, PNG, WEBP, tamaño máximo: 2 MB).



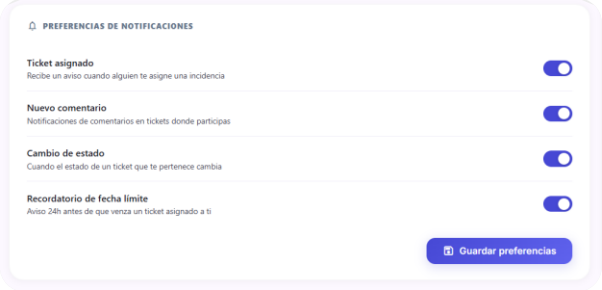
Seguridad

Presiona el botón “Enviar enlace de recuperación” para modificar tu contraseña actual.



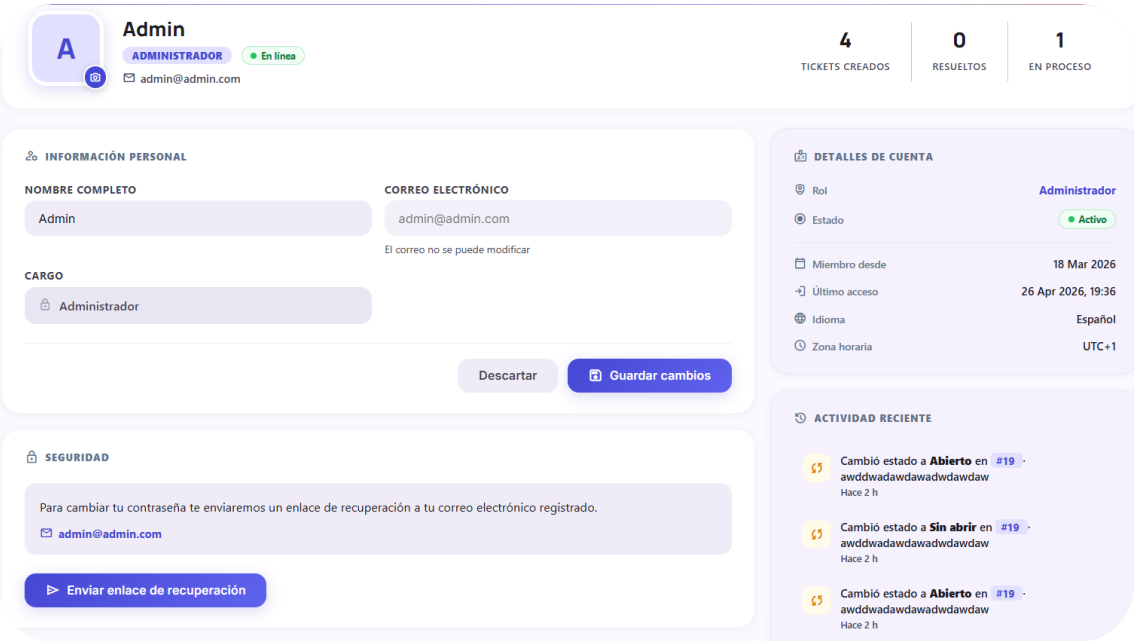
Preferencias de notificaciones:

Marca o desmarca las casillas según el tipo de notificaciones que quiera recibir.



Información de cuenta:

Consulta tus datos personales, detalles de la cuenta (rol, fecha de registro) y el historial de actividad reciente.

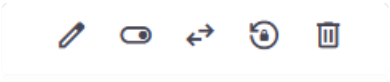


7.3.7 Configuración (solo accesible para administradores)

Gestión de usuarios

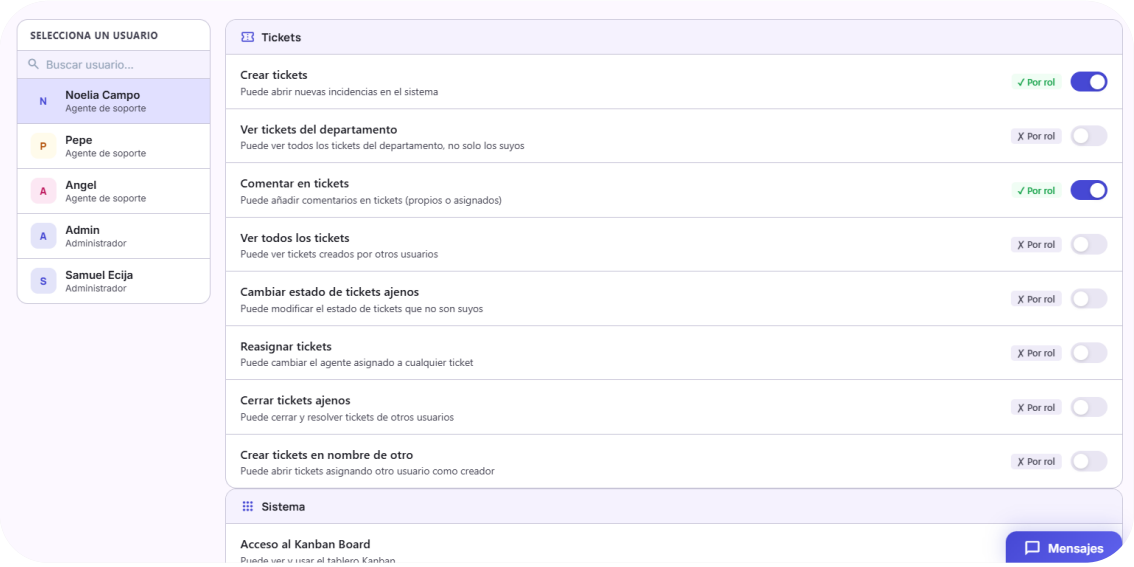
Tabla con todos los usuarios del sistema. Acciones disponibles por usuario:

- **Editar datos**: modifica nombre y email
- **Desactivar cuenta**: bloquea el acceso sin eliminar el usuario
- **Cambiar rol**: asigna rol admin, agente o cliente
- **Forzar cambio de contraseña**: obliga al usuario a cambiar su contraseña en el próximo login
- **Eliminar cuenta**: elimina el usuario permanentemente



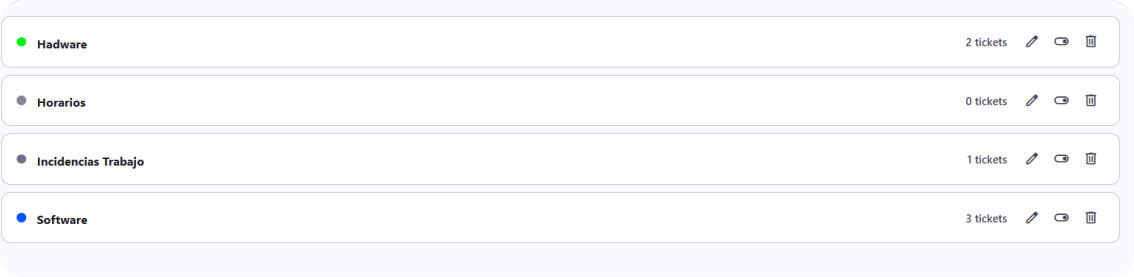
Permisos especiales:

Añade o revoca permisos individuales sin cambiar el rol del usuario.



Categorías de tickets

Crea, edita o desactiva las categorías disponibles al abrir un ticket. Cada categoría pertenece a un departamento.



Departamentos

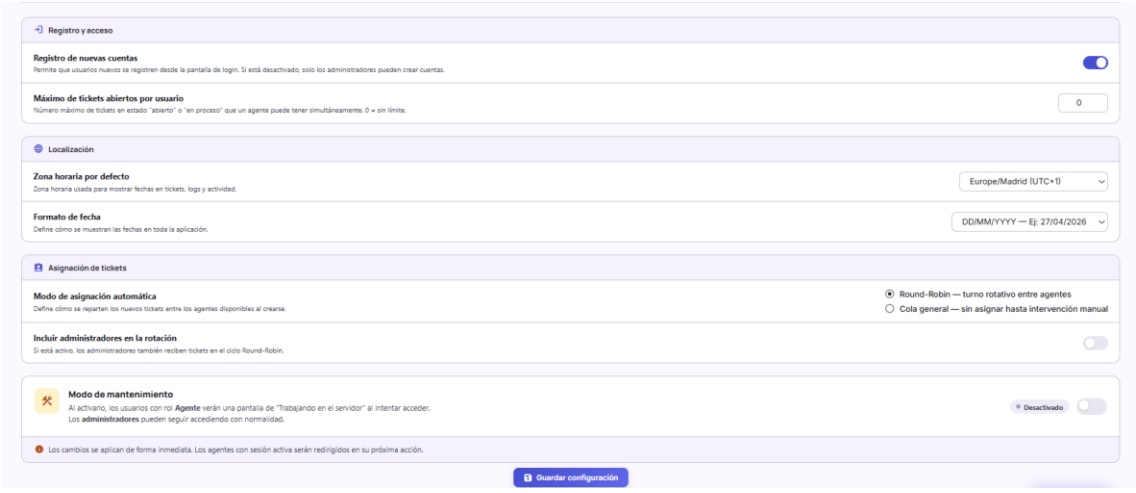
Crea y gestiona los departamentos de la organización. Desde cada departamento puedes:

- **Gestionar miembros**: añadir o quitar agentes del departamento.
- **Editar**: modificar el nombre del departamento.
- **Desactivar / Eliminar**: gestionar el estado del departamento.

Configuración del sistema

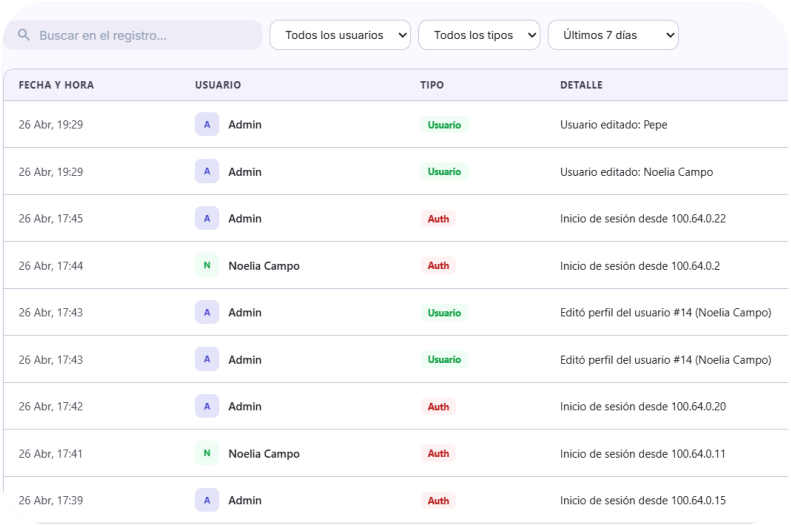
Modifica los parámetros globales de la aplicación:

Parámetro	Descripción
Registro abierto	Permite o bloquea el registro público de nuevos usuarios
Máx. tickets por usuario	Límite de tickets activos por usuario
Inactividad (minutos)	Tiempo sin actividad para marcar un usuario como inactivo
Zona horaria	Zona horaria del sistema
Formato de fecha	Formato de visualización de fechas
Modo de asignación	manual o round-robin (asignación automática al agente con menos tickets)
Incluir admins en rotación	Si los administradores participan en la rotación round-robin
Modo mantenimiento	Activa o desactiva el modo mantenimiento del sistema



Registro de actividad

Historial completo de todas las acciones realizadas en el sistema. Incluye buscador para filtrar por usuario o tipo de acción.



8. CONTROL DE CALIDAD Y PRUEBAS

8.1. Estructura del proyecto

El proyecto sigue una estructura modular basada en el patrón MVC, con separación clara entre capas:

- MVC/controllers/ controladores de cada módulo.
- MVC/models/ modelos con acceso a datos mediante PDO.
- MVC/views/ plantillas PHP por módulo.
- libs/ componentes del núcleo (FrontController, AppController, View, SPDO, Config, Csrf, Mailer).
- assets/ recursos estáticos (CSS, JS por módulo).
- tests/ tests unitarios con PHPUnit.
- database/ script SQL completo de instalación.
- .github/workflows/ pipeline de integración continua.

Esta separación garantiza que los cambios en una capa no afecten directamente a las demás y facilita la localización y corrección de errores.

8.2. Pruebas Automatizadas

El proyecto cuenta con integración de pruebas automáticas usando PHPUnit 11.5 configurados en phpunit.xml y ejecutados automáticamente en GitHub Actions (phpunit.yml).

Archivo	Qué se prueba
CsrfTest.php	Generación de tokens (64 chars hex), validación correcta/incorrecta, regeneración, campo HTML oculto.
PasswordValidationTest.php	Reglas de seguridad de contraseña: longitud mínima (8) y máxima (72), mayúsculas, minúsculas y dígitos requeridos.
UserPermissionDefaultsTest.php	Que los 3 roles (admin, user, cliente) tienen los 10 permisos definidos, que admin los tiene todos en true y cliente en false, y que los permisos son de tipo boolean.
TicketModelTest.php	Generación de la cláusula SQL de visibilidad según rol: admin sin filtro, agente sin departamentos, agente con departamentos.
ConfigTest.php	Que Config::db() devuelve el array correcto con las variables de entorno.

Para ejecutar los tests localmente:

vendor/bin/phpunit tests

8.3. Pruebas Manuales

Las pruebas manuales se realizaron navegando por todos los flujos principales de la aplicación y verificando el comportamiento esperado en cada caso:

Área	Casos probados
Autenticación	Login correcto, login con credenciales incorrectas, recuperación de contraseña, cambio obligatorio en primer login
Registro	Registro público con rol cliente, validación de campos, email duplicado
Tickets	Creación, edición, cambio de estado, asignación manual, asignación round-robin, adjuntos
Permisos	Verificación de acceso según rol (admin, agente, cliente) y overrides individuales
Kanban	Visualización de columnas por estado, acceso restringido por permiso
Configuración	Gestión de usuarios, categorías, departamentos, parámetros del sistema
Seguridad	Protección CSRF en formularios, acceso a rutas protegidas sin sesión, tokens de recuperación de un solo uso
Notificaciones	Recepción de notificaciones por asignación, comentario y cambio de estado

8.4. Dependencias

Las dependencias de producción se gestionan con Composer y son mínimas:

Paquete	Versión	Uso
phpmailer/phpmailer	^7.0	Envío de correos SMTP
vlucas/phpdotenv	5.0	Carga de variables de entorno
phpunit/phpunit	^11.5	Tests

8.5. Manejo de Errores

- Existe una vista específica errorView.php para mostrar errores de forma amigable al usuario.
- Los controladores validan todas las entradas antes de procesarlas y devuelven mensajes de error claros cuando la acción no está permitida o los datos son incorrectos.
- Las consultas a base de datos usan PDO con consultas preparadas en todos los modelos, previniendo inyección SQL.
- Las credenciales y configuración sensible nunca están en el código fuente, sino en el archivo .env excluido del repositorio mediante .gitignore.

8.6. Buenas Prácticas

- .gitignore excluye vendor/, .env, uploads/ y cachés de PHPUnit.
- Separación estricta entre lógica de negocio, presentación y control de flujo.
- Tokens CSRF en todos los formularios POST.
- Contraseñas almacenadas con hash bcrypt mediante password_hash().
- Sesiones protegidas con regeneración de ID en login.

9. PRESUPUESTO

9.1. Fases

9.1.1. Análisis y planificación - 1 semana

- Definición de requisitos funcionales y no funcionales.
- Análisis de herramientas existentes en el mercado.
- Planificación del proyecto y distribución de tareas.

9.1.2. Diseño y arquitectura - 1 semana

- Diseño del esquema de base de datos.
- Definición de la arquitectura MVC y estructura de carpetas.
- Diseño de la interfaz de usuario y prototipado inicial.

9.1.3. Desarrollo - 6 semanas

- Implementación del núcleo MVC.
- Desarrollo de los módulos de autenticación, tickets, permisos, notificaciones, mensajería, auditoría e interfaz.
- Integración de PHPMailer y phpdotenv.
- Implementación del sistema de seguridad.

9.1.4. Pruebas y control de calidad - 1 semana

- Escritura de tests unitarios con PHPUnit.
- Pruebas manuales de todos los flujos principales.
- Corrección de errores detectados.

9.1.5: Implementación y lanzamiento - 1 semana

- Despliegue en servidor de producción (Railway).
- Verificación del entorno de producción.
- Documentación final y memoria del proyecto.

El proyecto se ha desarrollado como trabajo académico. No ha existido coste económico directo en licencias ya que todo lo utilizado es open source, y el hosting fue de forma gratuita.

Fase	Horas
Análisis y diseño inicial	20 h
Diseño de base de datos	30 h
Desarrollo del núcleo MVC (framework)	100 h
Desarrollo módulo autenticación y seguridad	35 h
Desarrollo módulo de tickets	30 h
Desarrollo sistema de permisos	25 h
Desarrollo notificaciones y auditoría	20 h
Desarrollo interfaz	30 h
Integración y pruebas	30 h
Documentación y memoria	20 h
TOTAL	**340 horas**

Valorando el trabajo a un costo por hora de 15€ el coste estimado sería de unos 5100€.

10. CONCLUSIONES Y VIAS FUTURAS

El proyecto nos ha permitido aplicar de forma práctica e integrada los conocimientos adquiridos a lo largo del ciclo: PHP orientado a objetos, bases de datos relacionales, diseño web, seguridad web, gestión de sesiones y control de versiones.

La decisión de construir el framework MVC desde cero, aunque más costosa en tiempo, ha sido la más valiosa desde el punto de vista del aprendizaje: entendemos exactamente cómo funciona cada pieza del sistema porque la hemos escrito nosotros.

Lo que nos hubiera gustado hacer y no ha dado tiempo

- Envío de notificaciones vía WhatsApp Business API, se descartó por los requisitos de verificación de cuenta, los costes de la API y la falta de tiempo.
- App móvil para que los clientes consulten el estado de sus tickets.
- Sistema de SLA (acuerdos de nivel de servicio) con alertas automáticas por incumplimiento para los trabajadores de una empresa.

Vías futuras

- Despliegue en VPS con dominio propio
- Soporte multiidioma
- Búsqueda full-text en tickets
- Exportación de informes en PDF
- Sistema de SLA con vencimiento automático y alertas
- App móvil para clientes

Reflexión personal

Este proyecto ha supuesto el mayor reto técnico que hemos afrontado durante el ciclo. No solo por la cantidad de funcionalidades implementadas, sino por la responsabilidad de tomar decisiones de arquitectura que afectan a todo el sistema. Hemos aprendido que las decisiones tempranas como la estructura de permisos o el esquema de la base de datos son las más difíciles de cambiar después, y que documentar bien desde el principio ahorra mucho tiempo en las fases finales.

11. BIBLIOGRAFIA/WEBGRAFIA

Documentación oficial PHP: https://www.php.net/manual/es/

MariaDB Documentation: https://mariadb.com/kb/en/documentation/

Three.js Documentation: https://threejs.org/docs/

PHPMailer: https://github.com/PHPMailer/PHPMailer

PHP dotenv: https://github.com/vlucas/phpdotenv

OWASP Top Ten: https://owasp.org/www-project-top-ten/

MDN Web Docs: https://developer.mozilla.org/

MDN CSS: https://developer.mozilla.org/es/docs/Web/CSS

Stack Overflow: https://stackoverflow.com/

W3Schools PHP Tutorial: https://www.w3schools.com/php/

Apuntes del módulo de Desarrollo Web en Entorno Servidor.

Apuntes del módulo de Desarrollo Web en Entorno Cliente.